

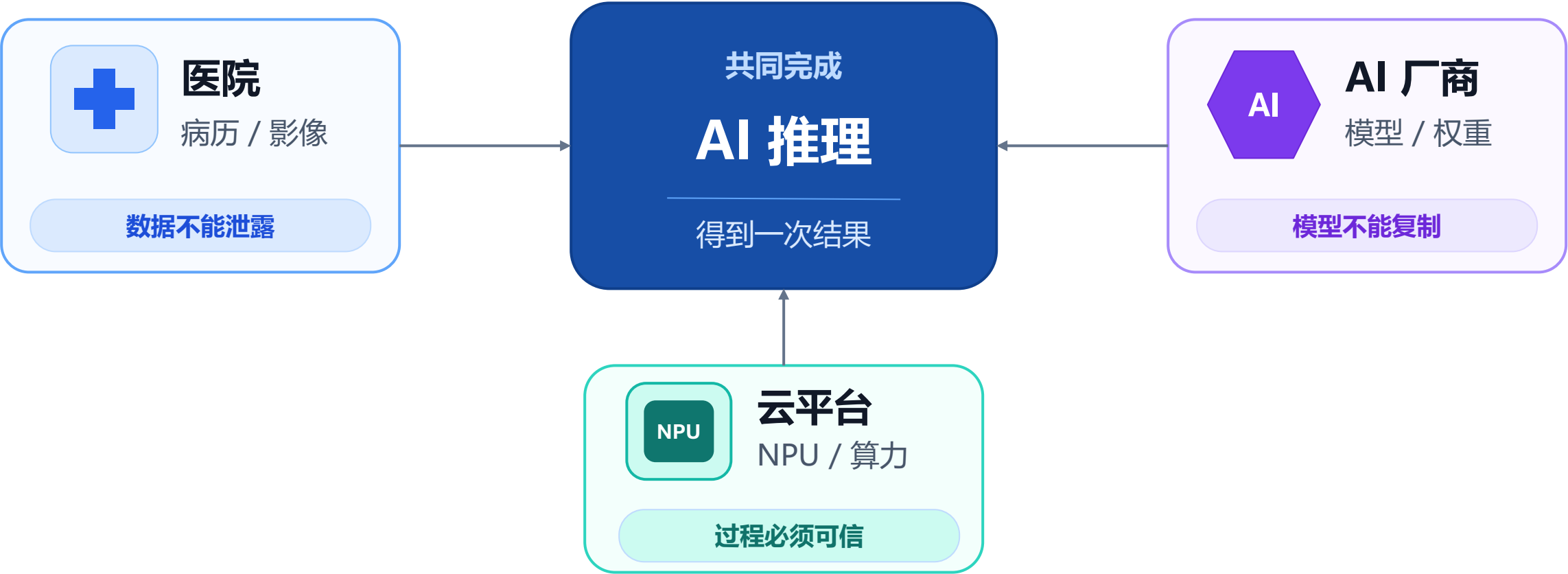
Crypto for AI

在不信任的世界里，实现可信的智能

CANN

数据、模型与算力，如何可信协作？

一次云端 AI 辅助诊断



彼此不能完全信任，却必须共同完成计算。



目录

Part 1 AI时代需要保护什么?

Part 2 密码学简史

Part 3 Crypto4AI – TEE

Part 4 Crypto4AI – FHE

Part 5 Crypto4AI – ZKP

Part 6 密码学SIG现状及规划

AI时代，密码学能保护什么？

为什么 AI 需要密码学？

传统应用场景：单点介入

- 访问验证：登录前证明身份与权限（口令、证书、API密钥等），验过才放行。
- 传输加解密：HTTPS/TLS 保护链路上的数据，路上不被偷看或篡改；到端解密后业务照常处理。

共同点：密码学不参与业务，只在门禁或传输期间提供安全能力

AI：具体场景里的安全问题

- 云端推理：提示词/样本不想被服务端看见 → 密态计算
- 联合建模：数据不能外发 → 多方安全计算
- 托管推理：是否按约定模型执行 → 可验证推理
- Agent/生成内容：越权与出处不清 → 签名与凭证

密码学用到 AI 的哪些方面？

输入与提示

样本、提示词、检索内容进模型/云端时怎么提供保护，不被窃听、篡改、截存

- 安全协议

多方数据协作

如何合规使用多方敏感数据训练模型并保护隐私

- 安全多方计算
- 差分隐私

模型与推理过程

如何证明云端运行的是特定模型、如何保护核心模型资产不被窃取、计算过程是否可信

- 零知识证明

权限与内容出处

Agent能调用什么、如何验证AI生成内容（反深伪/溯源）

- 数字签名
- 哈希函数

现有大部分应用场景中，密码学主要扮演门禁与邮差的角色，只在访问、传输、校验那一刻介入。在涉及到一些新场景（包括AI）时，密码学需要开始参与具体业务，跟随计算任务一起跑，不再只是单纯门卫



工业界/学术界针对相关技术已有的研究或部署

业界研究现状：提性能、降开销；部分密码学方法已具备可落地的可行性方案

密码学方法	保护对象	安全目标	工业进展	阶段
同态加密	密态推理中的输入	计算过程中明文不可见	Concrete ML 等编译器已经开源；加速是主要问题	产品化推进中
安全多方计算	联合建模中的各方数据	任何一方无法获得完整输入	CrypTen 以 PyTorch 接口提供安全多方计算	已有开源工具
零知识证明	推理过程与模型权重	权重不必公开，执行过程可验证	zkLLM 已对 13B 模型生成证明；时延仍需降低	已有规模化结果
数字签名	调用权限与内容来源	委托可验证、清单可验证	C2PA 已进入主要产品；Agent 委托标准仍在起草	已进入产品

当前一些主流研究方向

- 同态加密**：开发大模型专属的硬件加速芯片（如FPGA/ASIC）与近似计算算法，扩大可支持的模型，降低算力开销和性能衰减
- 安全多方计算**：设计混合协议，提升大模型协同训练的吞吐量，从统计与浅层模型扩到更深的网络
- 零知识证明**：以低成本、低时延实现大模型推理与出处的可验证性
- 数字签名**：结合去中心化身份与C2PA（内容真实性）国际标准，为Agent跨平台API授权以及多模态AI生成内容（AIGC）提供端到端的防篡改溯源

每类方法都有明确的产业所关心的问题；关键在于如何做到可商用、可部署



同态加密与安全多方计算：解决密态推理与联合建模

在AI workflows中，输入、提示词、检索内容等是隐私泄露的最高发地带。这些数据一旦进入云端或模型，往往包含用户私密意图、企业商业机密或合规敏感数据

同态加密

保护点：云端推理的样本、提示词、检索内容——进模型前不以明文存在服务端

风险点：服务端偷看、篡改、留存、拿去做二次训练

可行方案：用户将样本或提示词加密后发送到云端，模型在密文上完成推理，服务端无法获得明文

业界案例：Zama公司推出Concrete ML：将scikit-learn、树模型、神经网络以及大语言模型转换为可在密文上执行的电路。大语言模型采用HybridFHE，线性层在云端对密文计算，非线性部分在客户端完成

- **案例现状：**GPT-2 密态 1 token 单GPU≈11s；树模型密文计算比明文浮点数计算慢 10^3-10^4 倍

对于多方数据协作，如何在保护隐私的前提下，利用多方敏感数据共同训练模型或进行联合分析

安全多方计算

保护点：联合建模、联合统计时各方的原始数据与数据控制权

风险点：参与方共谋恶意串通或恶意输入篡改的攻击，或数据被单方拿走

可行方案：秘密分享 + 交互协议——一起算，谁都拿不到完整输入

业界案例：Meta AI发布的CrypTen技术框架，通过多方安全计算中的秘密分享协议，将矩阵乘法、非线性激活函数以及反向传播自动求导全部转换为分布式的密文多方协作计算方案

- **案例现状：**密文音频预测的推理速度可超越实时音频播放速度 (Faster than real-time)

两种技术都有产业工具；共性难题是计算能力和算力开销



零知识证明：在不公开权重的前提下验证推理

核心逻辑：用数学上的承诺绑定模型，用密文电路演算推理，最后只证明计算过程的正确性，不泄露隐私数据

1. 模型承诺

云端将模型权重压缩锁死为一个公开的密码学指纹，防止模型被暗中调包或篡改

2. 私有输入

用户数据作为隐私变量注入系统，全程对外部验证者保持隐蔽

3. 电路化执行

将矩阵乘法与Attention等运算编译为数学方程组电路，在密文状态下推理

4. 输出与证明

生成AI结果及一份极小的数学证明；验证者无需重跑大模型，仅凭证明即可秒级确认推理的真实性

工业进展

- zkLLM (CCS 2024)：对 OPT-13B / LLaMA-2-13B 的完整推理生成证明，证明大小小于 200kB，表明大模型上的可验证推理已经实现
- zkPyTorch (ePrint 2025/535)：将 PyTorch 模型编译为电路，降低使用门槛
- EZKL：开源系统，输入为 ONNX 计算图，输出为 zk-SNARK。对单次推理进行核验已经可以使用

待解决的问题

- 将证明时延从分钟量级降到可实际使用的水平（Llama-3 的公开数据约为 150 秒/token，可作为当前基准）
- 使工程师不必手工编写电路
- 也可与同态加密配合使用：分别保护输入机密性与执行过程的可验证性

大模型上的可验证推理已经实现；接下来是降低时延、和方便非密码学专业出身的AI工程师使用

数字签名：Agent授权鉴别与生成内容的来源验证

AI Agent的合规行为、AIGC 的内容真实性，均统一构建于数字签名的密码学底座之上

场景一：AI Agent跨服务调用鉴权

- **安全风险（权限蔓延与越权滥用）**：例如：Agent 被诱导利用高阶凭证，将本属于隐私的日历日程（A服务）暗中通过邮件（B服务）外发
- **业界现状**：缺乏认证机制。Knostic 扫描发现公网暴露近 2000 台 MCP 服务器，抽样资产的工具列表接口（tools/list）均未做鉴权
- **密码学解决方案（动态能力凭证Capability）**：调用的最终权限由“人类用户、Agent、当前任务”三者签名取交集，在调用链中传递且权限只减不增
- **前沿标准**：有一些提交给诸如IETF的草案，尚未形成RFC标准

核心验证能力：验证用户真实意图、Agent 代理身份、以及当前单次任务的动态授权范围

场景二：生成内容的来源

- **安全风险（伪造/深伪）**：无法确定互联网上的图像、音视频或文本是否确实由宣称的 AI 模型原厂签发，且无法确定该文件在流转过程中是否经过二次恶意篡改或剪辑
- **密码学解决方案（C2PA 签名内容绑定）**：依托C2PA国际标准，在模型生成内容的瞬间，利用原厂私钥进行数字签名，将生成工具、时间戳及修改历史通过哈希函数机制强绑定，封装为一份防篡改的内容凭证
- **主流产品实践**：OpenAI在Sora等生成式模型中，接入 C2PA Content Credentials（内容凭证）体系

核心验证能力：验证内容签发者（模型厂商）的真实身份、以及多媒体资产自生成后是否遭遇任何像素级篡改

C2PA标准已在内容端成熟落地，而关于AI Agent多级嵌套授权与凭证传递的标准仍在讨论/制定中

出处 Knostic <https://knostic.ai/blog/mapping-mcp-servers-study> · HDP <https://datatracker.ietf.org/doc/draft-helixar-hdp-agentic-delegation/>
Intent Token <https://datatracker.ietf.org/doc/draft-williams-intent-token/> · C2PA <https://c2pa.org> · OpenAI C2PA <https://openai.com/index/advancing-content-provenance/>



结语：保护对象已由通信信道扩展到AI运行过程与模型等场景

面向人工智能的密码学，要保护的是系统中一旦向外提供就难以撤回的对象：输入数据、模型参数、调用权限与内容来源

密码学的角色/定位有所改变

- AI应用场景下，密码学需要深入业务——推理、联合建模、托管执行、Agent与生成内容——解决安全问题
- 模型部署到云端之后，需要保护的不再只是传输中的数据。训练样本、提示词、检索库、模型权重、Agent 的调用权限，以及生成内容的来源，都成为密码学应当覆盖的对象

密码学方法应用场景

对应的常用手段仍是同态加密、安全多方计算、零知识证明和数字签名；这些并不是为今天的大模型新造出来的，值得研究的是如何把它们嵌进训练、推理等AI应用的流程，在提供高效处理能力的情况下不影响AI业务运行

密码学方法应用的难点/研究方向在于如何更高效地服务于人工智能

针对系统中的某一类对象，如何做到实际部署；应当先确定要保护什么，再选择相应的密码学方法，在已有实践上继续深入。

需要保护的对象可能在业务里；效率、适用范围和协议上尚未解决的问题，就是潜在的研究课题

目录

Part 1 AI时代需要保护什么？

Part 2 密码学简史

Part 3 Crypto4AI – TEE

Part 4 Crypto4AI – FHE

Part 5 Crypto4AI – ZKP

Part 6 密码学SIG现状及规划

密码学简史

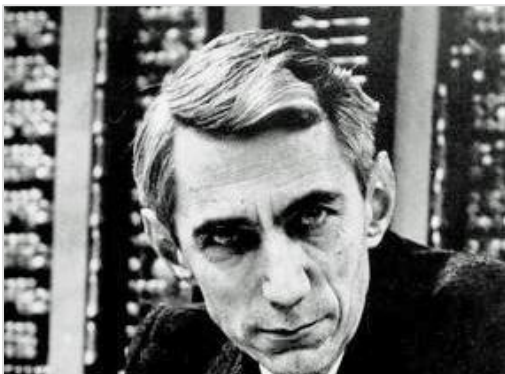


古典密码

约前 5 世纪 — 1949

密码棒
凯撒
虎符

用规则保护消息内容

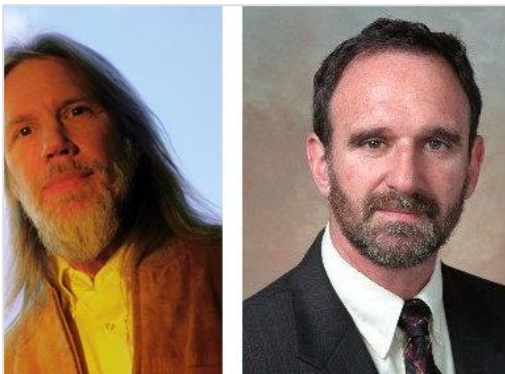


理论密码

1949 — 1975

香农
完美保密
混淆与扩散

安全可定义、可分析

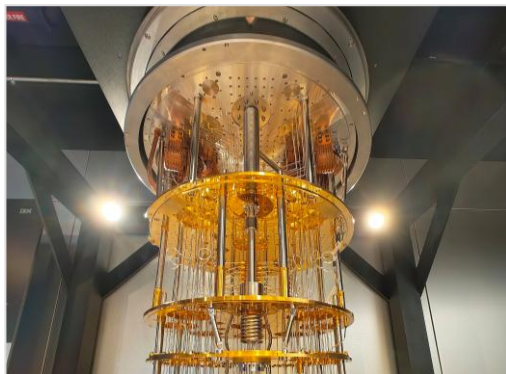


现代密码

1976 — 2008

公钥密码
密码协议
标准与基础设施

在开放网络中建立信任



保护计算

2009 — 至今

同态加密
可验证计算
可信执行

数据使用时仍受保护

斯巴达密码棒

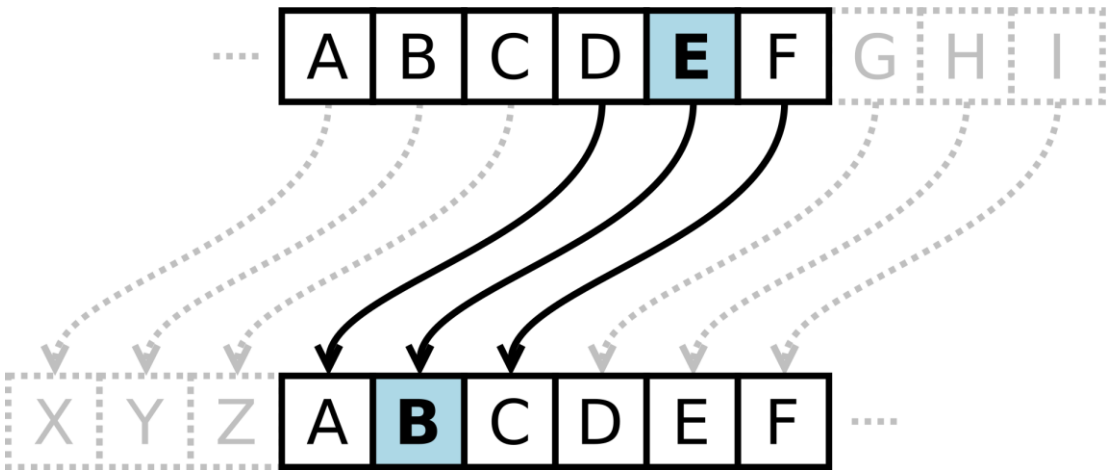
置换 · 改变位置



皮条展开后字符顺序被打乱；
只有使用匹配棒径和缠绕方式，字符才能重新对齐。

凯撒密码

代换 · 改变符号



每个字母按固定偏移替换。凯撒本人挪3位，一共只有25种可能。
规则简单、密钥空间有限，穷举即可恢复明文。

凯撒密码

T R O O P



M K H H I

后退 7 位 (k = 19)



战国青铜虎符 两半合上，才能调兵

虎 符

战国

铜虎剖成两半，君王一半、将领一半。合得上，才能调兵。

→ 合符验真

阴 书

《六韬》

一封密信剖成三份，三个人分头送。三份凑齐，才读得出。

→ 消息分片

字 验

《武经总要》

拿一首四十字的诗当密码本，一个字对一条军情，一战一换。

→ 共享码本

9 世纪 · 破

统计字符出现频率



阿尔-金迪

以英文为例：E、T 等字母频率较高
单表代换会保留语言统计特征



单表代换从此全线失守

密码分析成为可以总结和传授的方法

16 世纪 · 守

密钥控制多张代换表



阿尔伯蒂密码盘 (约 1467)

转一下盘，就换一张代换表。

多表代换削弱单一字母频率特征；
阿尔-金迪使用的统计字符。

1553 年贝拉索给它加上了可更换的密钥——维吉尼亚密码。

周期性密钥仍会留下重复结构

1863 年卡西斯基指出：密钥会重复，重复就是破绽。

Enigma：机器放大密钥空间



德军 Enigma 密码机

转子、起始位置和插线板共同形成庞大的设置空间

$$1.59 \times 10^{20}$$

也就是 1.59 万亿亿 —— 每秒试一百万个，要试上五百万年

60

5 个转子里选 3 个
还要排顺序

17,576

三个转子的
起始位置 26^3

1.5×10^{14}

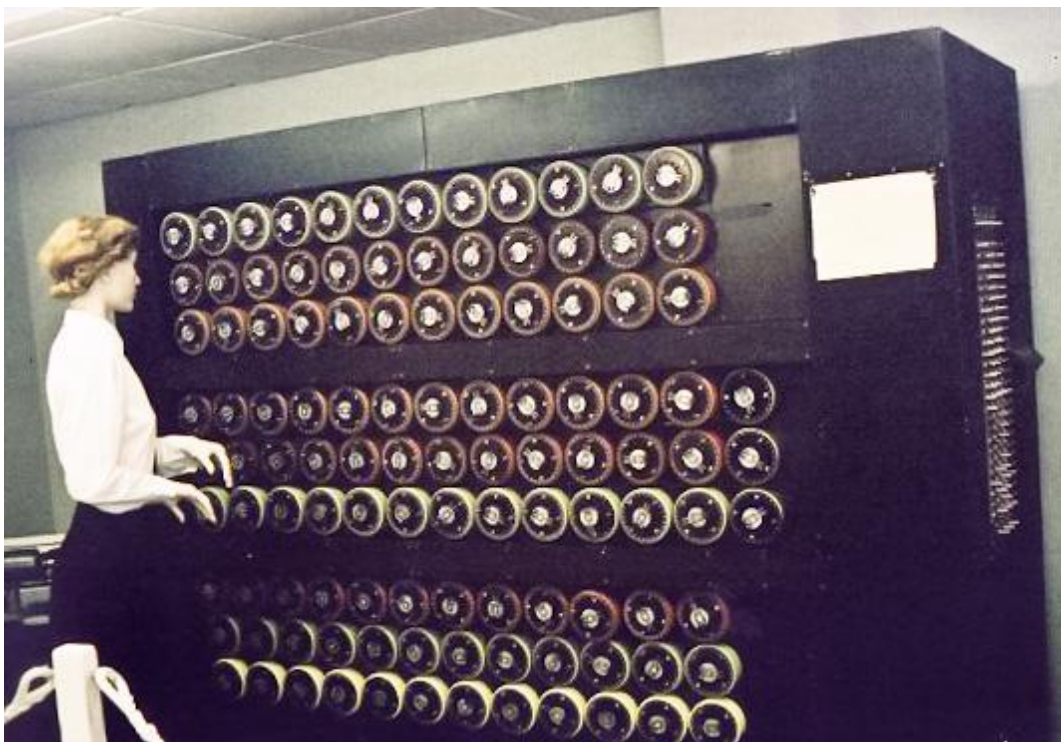
插线板上
十根跳线的接法



每次按键都会推动转子，使字符映射持续变化 —— 每按一个键，转子就走一格，代换表跟着换一张。

Bombe: 机器排除不可能

第一阶段 · 古典密码



英国 Bombe 搜索机 · 布莱切利园

1932

雷耶夫斯基

用置换理论刻画 Enigma 的内部结构，重建关键转子接线。

1939

图灵

用天气报告、固定用语这些已知明文构造约束，靠逻辑矛盾排除错误候选。

1940

Bombe

把候选设置的检验机械化，大幅压缩真正需要检查的范围。

关键转折 复杂不等于安全。现代密码学真正要的不是更复杂的机器，而是能定义、能证明安全的方法。

CANN



香农 · 《保密系统的通信理论》1949

完善保密性

香农给出严格定义，并说明一次一密在相应条件下可以达到完善保密。

混淆与扩散

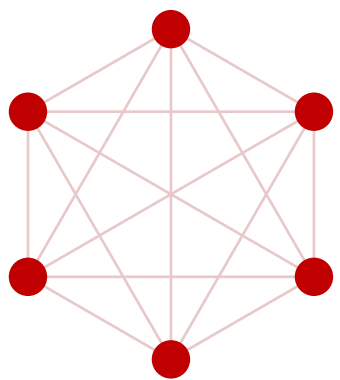
为分组密码设计提供基本思想，影响了 DES、AES、SM4 等算法。

从经验走向模型

安全性开始通过数学模型、攻击能力与假设进行分析。

千年难题

通信双方如何安全
获得共享密钥？



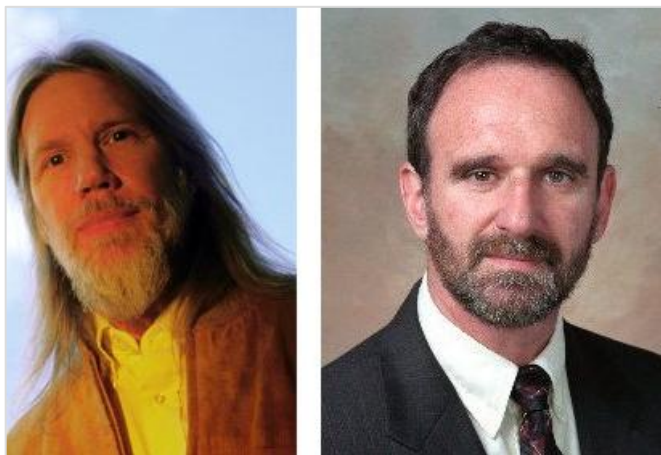
6 人

15把

1000 人

499,500把

两两之间预先共享一把密钥，用户数增长会带来近似**平方级**的密钥管理成本。



1976

无需预先共享同一秘密

Diffie-Hellman 通过公开交互协商共享密钥，无需预先秘密传递同一把密钥。



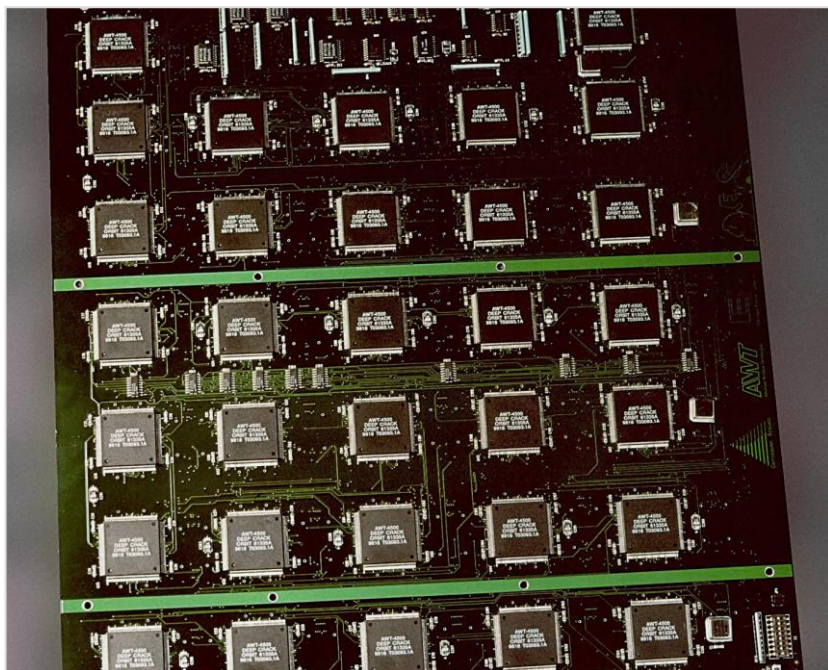
1977

公钥加密与签名

RSA 三人组给出实用的公钥加密与签名方案，推动身份认证和开放网络交易发展。

从 DES 到 AES：密码标准的演进

第三阶段 · 现代密码



1998 年，电子前哨基金会EFF花不到 25 万美元造了这台机器，56 小时破掉了美国的国家标准。

破解 DES 需要多久

1997 志愿者联合计算

96 天

1998 distributed.net

39 天

1998 EFF Deep Crack

56 小时

1999 专用硬件 + 十万台 PC

22 小时

2001 年 AES Rijndael 于 2000 年被 NIST 选中，2001 年正式成为 **AES 标准**；**公开征集与评审成为重要范式。**

CANN

密码战争：密码学成为公共基础设施

第三阶段 · 现代密码



1991

PGP



强加密进入个人计算机，普通用户开始拥有保护通信的工具。

1993

Clipper 芯片



通过密钥托管为政府保留合法访问能力，引发安全与信任争议。

2013

斯诺登



大规模监控披露后，“默认加密”逐渐成为互联网服务的共识。

结果 HTTPS 从 2013 年的不到三成，涨到今天 Chrome 里的 **95%** 以上。

CANN

HTTPS：一次安全连接如何建立

HTTPS = HTTP over TLS —— TLS 将密钥协商、身份认证和数据保护组合成一条安全连接

随便打开一个网站，按 F12 → Security:

Origin

 <https://gitcode.com>

[View requests in Network Panel](#)

Connection

Protocol	TLS 1.2
Key exchange	ECDHE_RSA with X25519
Server signature	RSA-PSS with SHA-256
Cipher	AES_256_GCM

真实连接示例：<https://gitcode.com>

- ① 协商会话密钥

ECDHE_RSA with X25519

经典椭圆曲线密钥协商，生成本次连接使用的共享密钥。
- ② 验证服务器身份

RSA-PSS with SHA-256

浏览器结合数字证书与数字签名，确认服务器没有被冒充。
- ③ 加密并认证数据

AES_256_GCM

使用会话密钥保护后续通信，同时提供机密性与完整性。

当这些算法被组合进 TLS，密码学就从理论与标准，真正变成了互联网基础设施。



从保护通信，到保护计算



1979
Shamir

图灵奖 2002

秘密共享

将秘密分成 n 份，达到门限 k 后才能恢复。



1982
姚期智

图灵奖 2000

安全多方计算

各方不公开自己的输入，也能共同得到计算结果。



1985
Goldwasser

图灵奖 2012

零知识证明

证明我知道答案，却不泄露答案本身。



1985
Micali

图灵奖 2012

零知识证明

与 Goldwasser、Rackoff 共同建立零知识证明的严格定义

四个人，三届图灵奖 秘密无需公开，计算仍可协作，结论依然能够验证——密码学开始保护计算本身。





传输中
密文

TLS全程加密



存储中
密文

磁盘、数据库加密



计算中
明文

传统计算必须解密才能算

← AI里最值钱的东西
数据、模型与请求
都需要保护。

模型权重、训练数据、推理请求 —— 传统 AI 计算中通常需要在执行阶段以**可处理形式**出现。

可信执行环境 TEE

在隔离环境中保护执行

→ Part 3

同态加密 FHE

在密文上完成特定计算

→ Part 4

零知识证明 · 多方计算

ZKP验证结论；MPC 保护多方输入

→ Part 5



目录

Part 1 AI时代需要保护什么?

Part 2 密码学简史

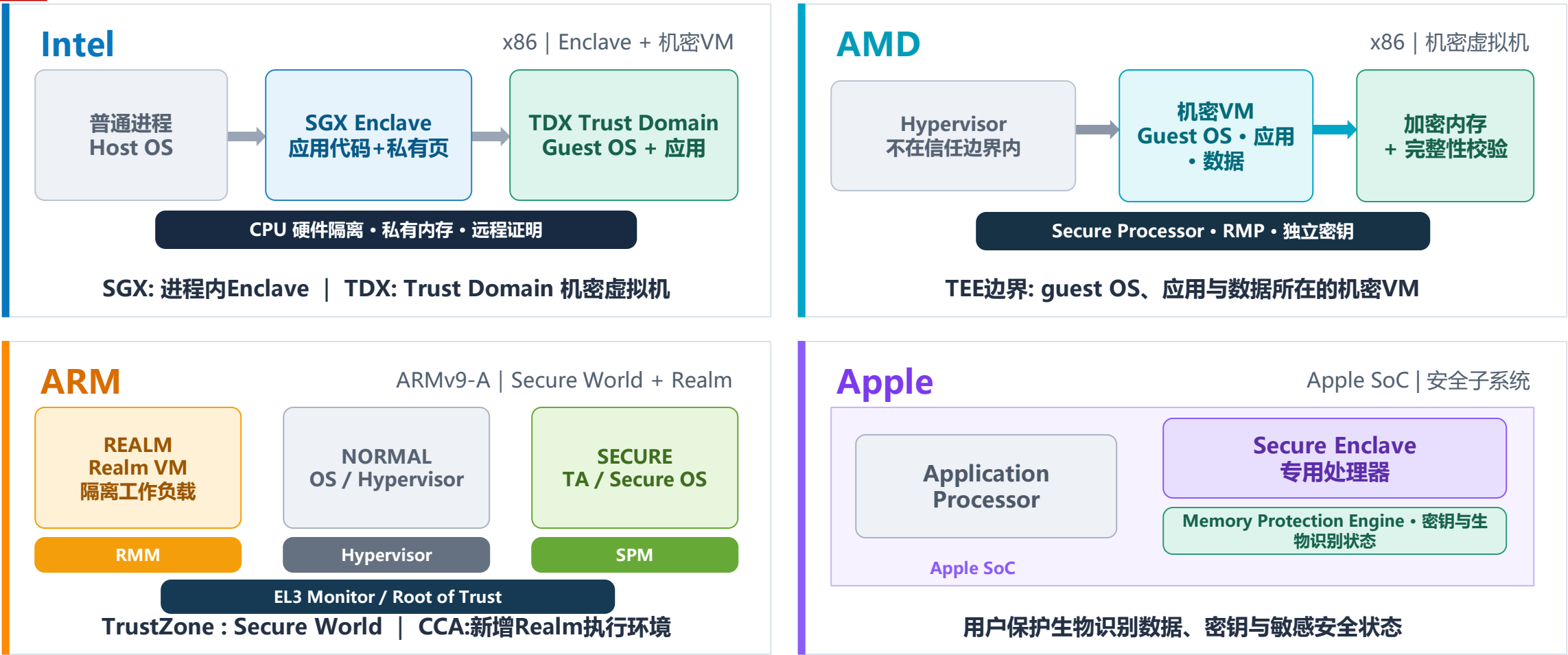
Part 3 Crypto4AI – TEE

Part 4 Crypto4AI – FHE

Part 5 Crypto4AI – ZKP

Part 6 密码学SIG现状及规划

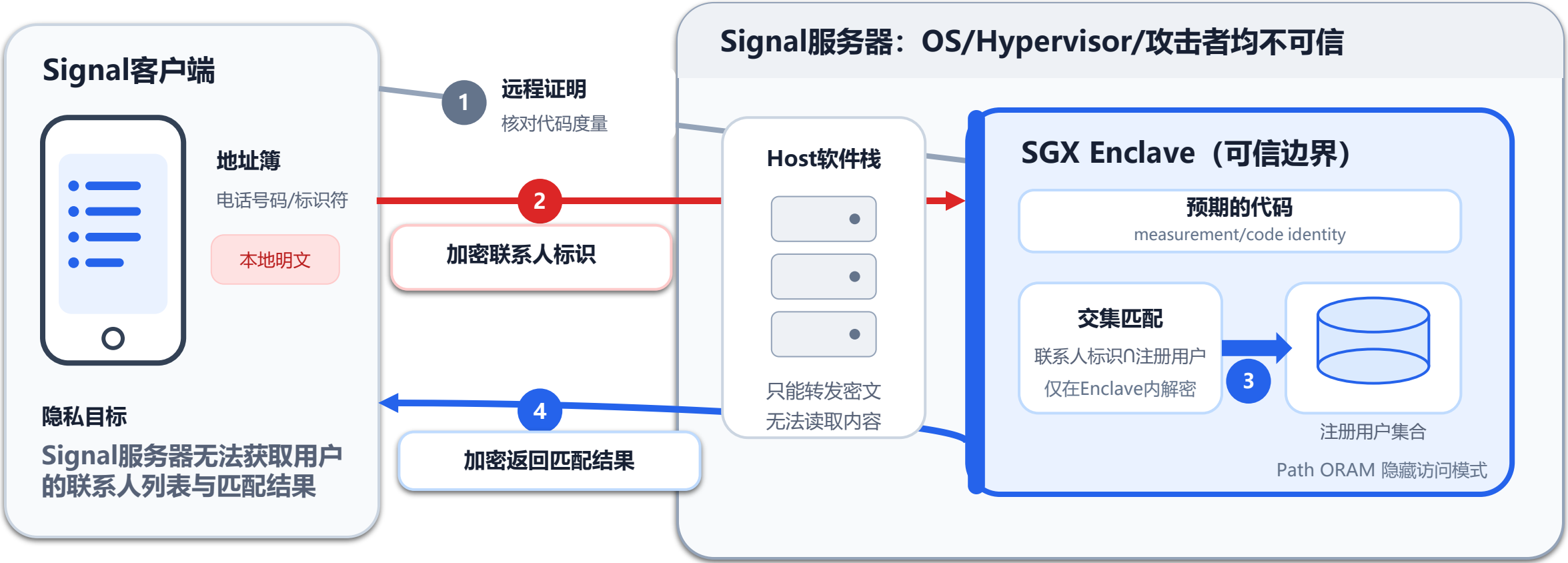
TEE—基于硬件隔离的重要安全基础设施



利用硬件建立一个比普通操作系统权限更高的、软件也不能随意读取或修改的执行边界

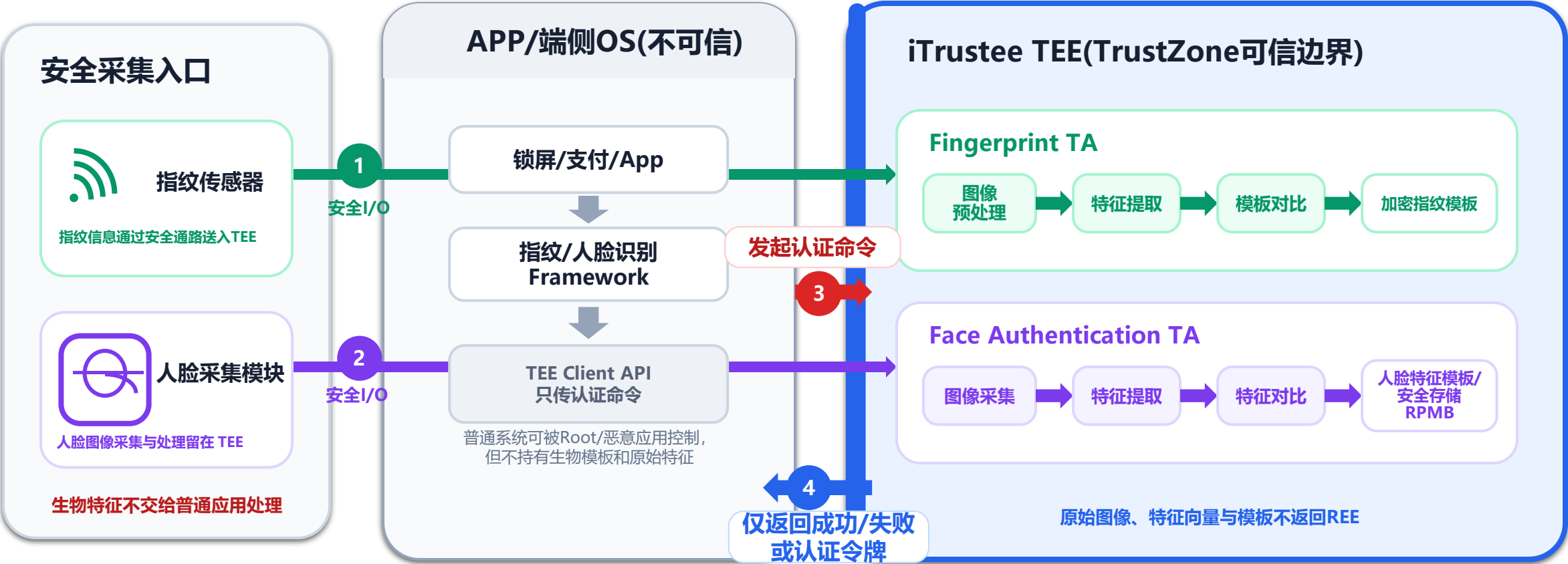
TEE如何保护Signal的私密联系人

客户端验证服务器端SGX Enclave，并让联系人查询始终以密文穿过不可信服务器。



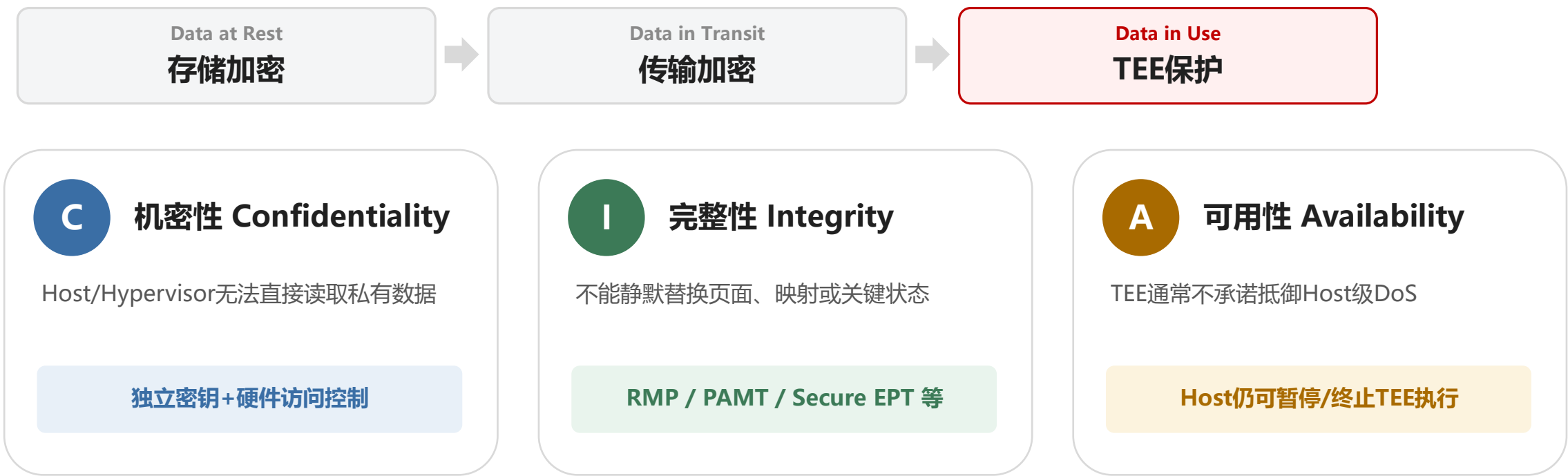
TEE在保护什么: 联系人标识、匹配结果和指定代码的执行过程不被OS/Hypervisor/攻击者获取

iTrustee把指纹与人脸认证留在TEE内



TEE在保护什么: 生物特征的采集、提取、模板对比和安全存储。

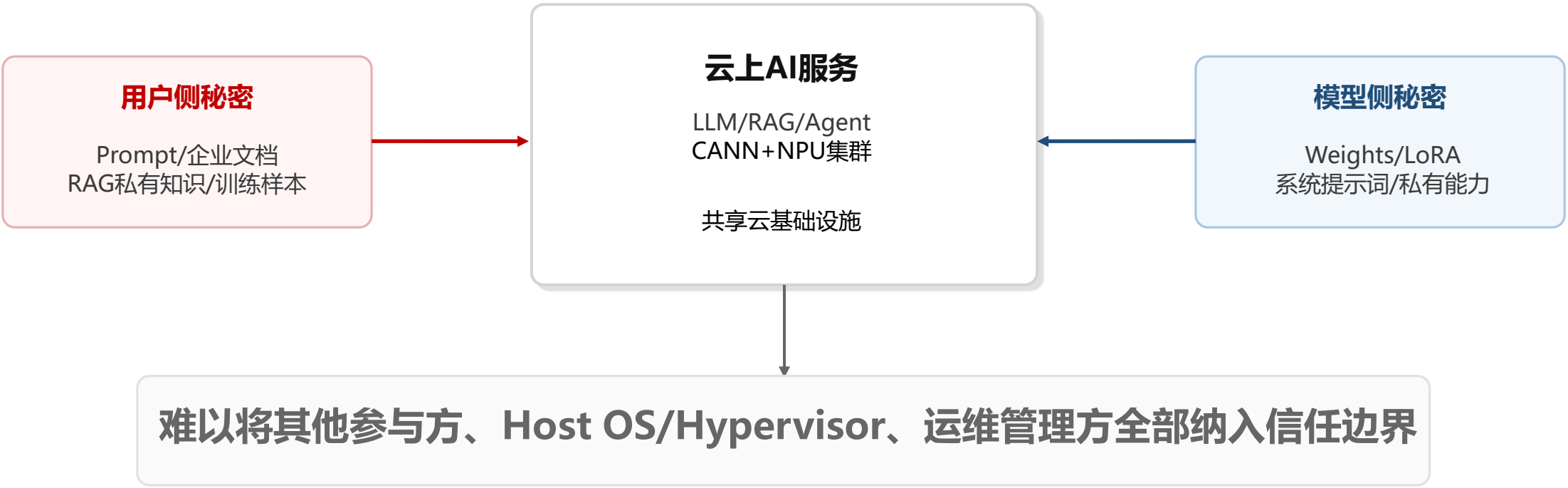
TEE的目标：保护Data in Use



远程证明：让远程用户知道连接了谁、验证对方正在运行的平台和初始代码，并据此决定是否释放密钥。

共享算力下的AI信任问题

问题： 如何在使用共享算力的同时，不把敏感信息完全暴露给云服务器？



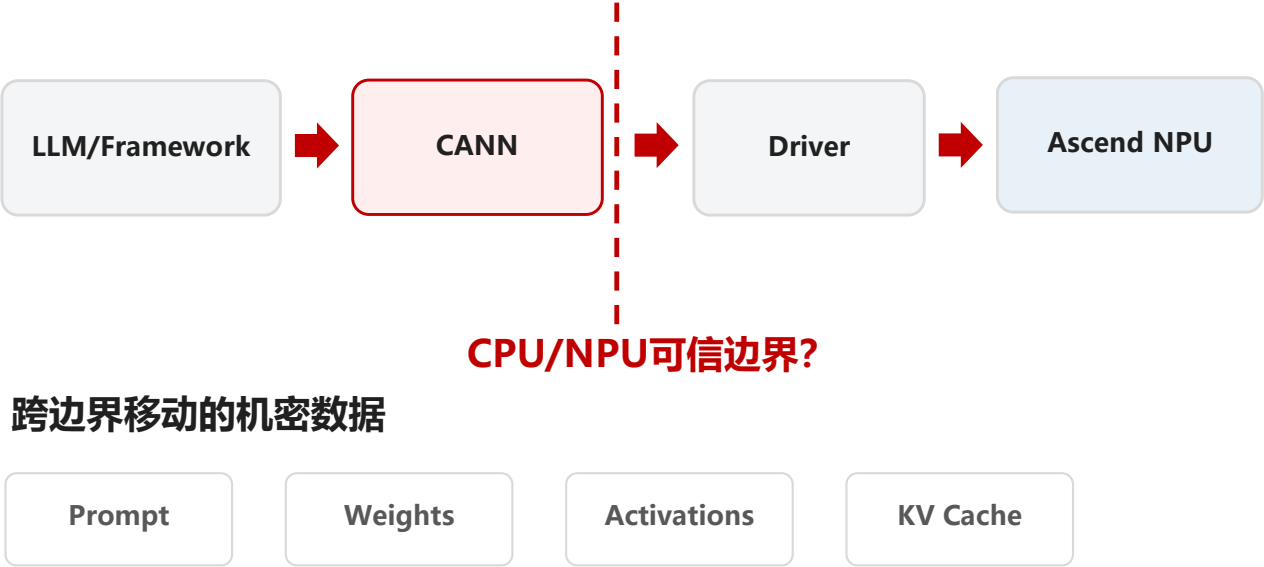
目标： 让敏感用户输入与私有模型能够参与计算，同时在使用阶段不暴露给不可信基础设施。

扩展的保护边界：CPU → NPU

传统工作负载



AI/LLM异构执行链



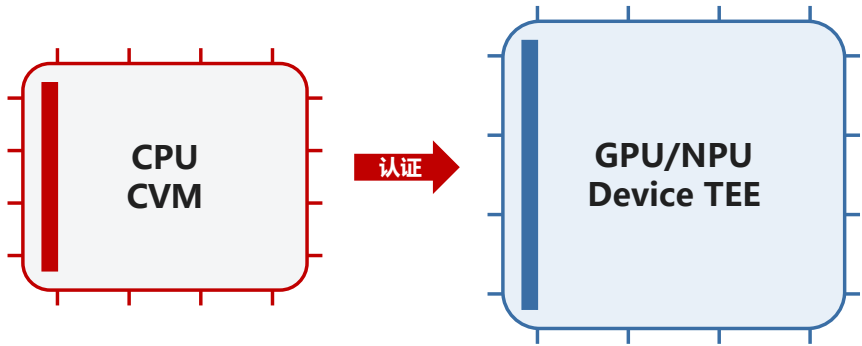
问题：数据离开CPU TEE进入NPU后，如何继续保证机密性与完整性，并建立可信的设备认证？
目标：把保护边界从一个处理器内部扩展成跨 CPU、互连和加速器的端到端问题

异构加速器的两类可信化路线

路线一

设备原生机密计算

在GPU/NPU内部增加安全启动、设备认证、受保护内存与可信执行上下文

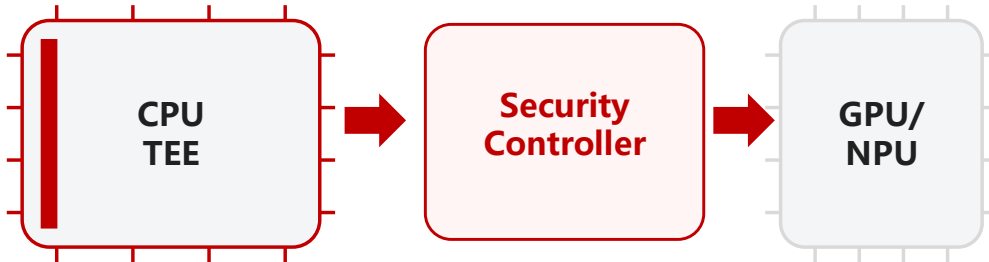


优点：设备侧可提供安全启动、隔离与远程认证
挑战：依赖硬件支持，并需与CPU侧TEE及可信I/O协同

路线二

系统级扩展保护

利用安全控制器、IOMMU、可信Runtime等把普通加速器纳入更大的受保护环境



优点：可复用缺少原生TEE能力的加速器
挑战：需要额外可信控制组件，并安全管理设备归属与I/O路径

两类方案都需解决设备隔离、通信保护和可信 I/O；核心区别在于，加速器信任由系统外部建立，还是由设备自身提供。

CANN为何进入安全关键路径



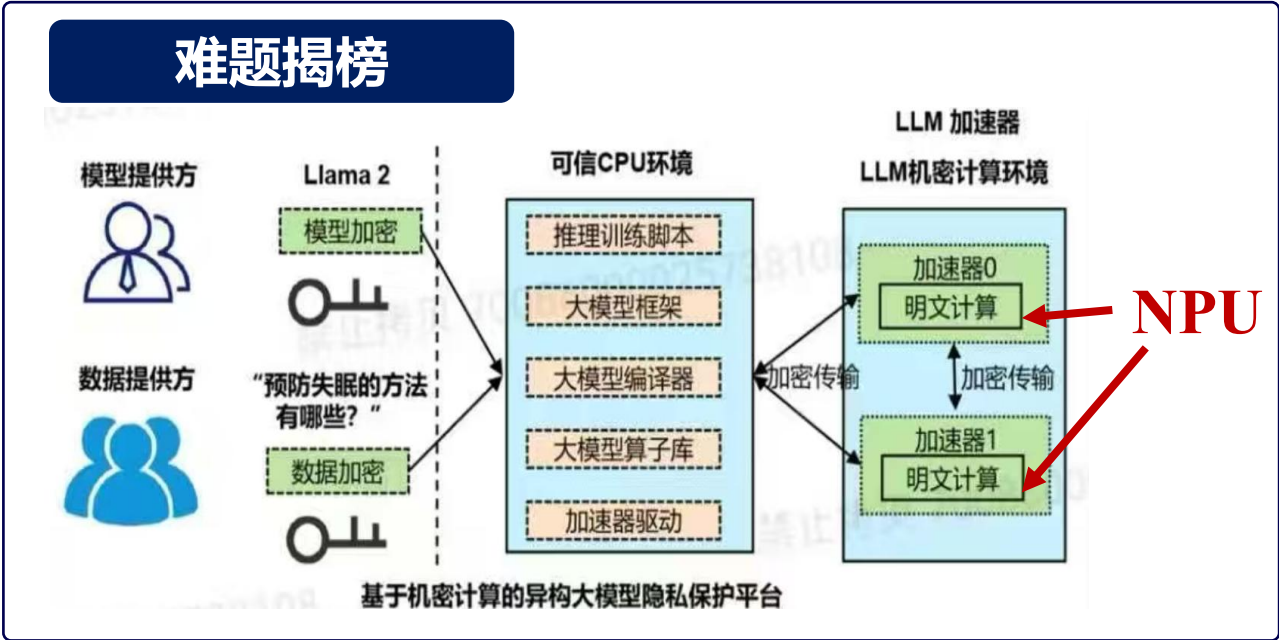
CANN位于“模型想计算什么”和“设备实际怎样计算”之间

- 1 图与算子转换**
将上层模型计算转换、优化为NPU可执行任务
- 2 运行时资源管理**
管理设备资源、Stream、任务下发与执行依赖
- 3 多设备协同**
组织多NPU通信与分布式执行

- 1 信任边界跨设备**
敏感数据从CPU进入互连、共享缓冲区和NPU设备内存，安全属性跨越多个硬件域。
- 2 可信计算基难界定**
依赖AI框架、CANN、Driver、Firmware与算子库，需要关注更多可配置、加载和更新的组件。
- 3 运行状态高度动态**
Graph、Descriptor、Stream、KV Cache 和 Buffer 的生命周期持续变化。
- 4 安全机制需要兼顾性能**
AI负载对带宽、延迟高度敏感，TEE设计需要兼顾安全保护与异构计算性能。

面向分布式大模型训推系统的加密方案

难 分布式大模型训练推理主要在GPU/NPU上执行，卡间通信数据加密性能成为瓶颈



- 现有密码标准算法运算组件在GPU/NPU上性能差，无法满足大模型卡间加密通信性能要求
- 难题要求：基准AES性能降低>15%，AES优化目标<5%

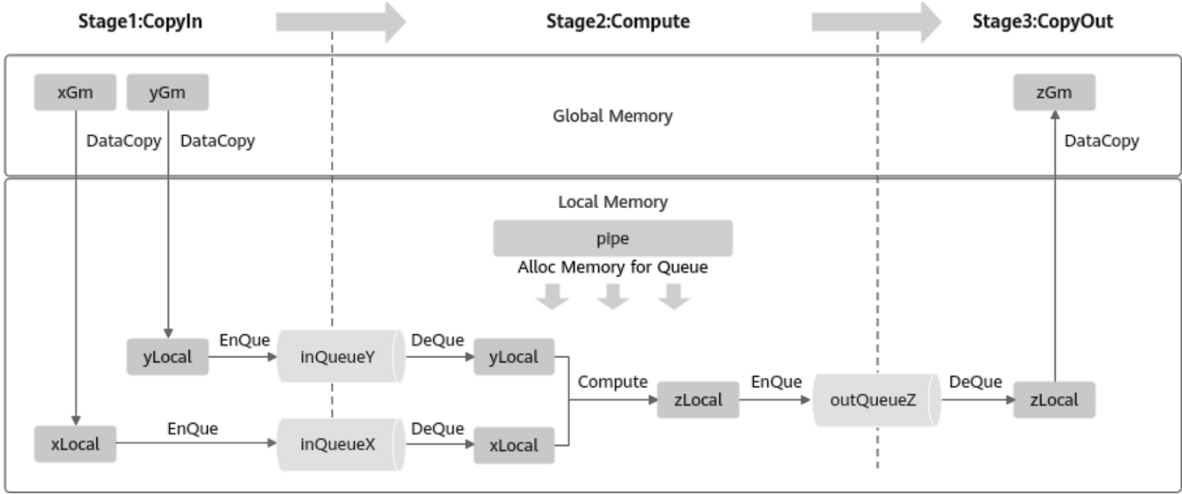
面向NPU的密码算法组件优化

- 充分利用**NPU高性能计算单元**提速：向量实现多路并行，矩阵运算实现查表
- 针对NPU中的多个独立计算单元，开展流水线路径优化设计，降低整体延迟

高性能计算单元

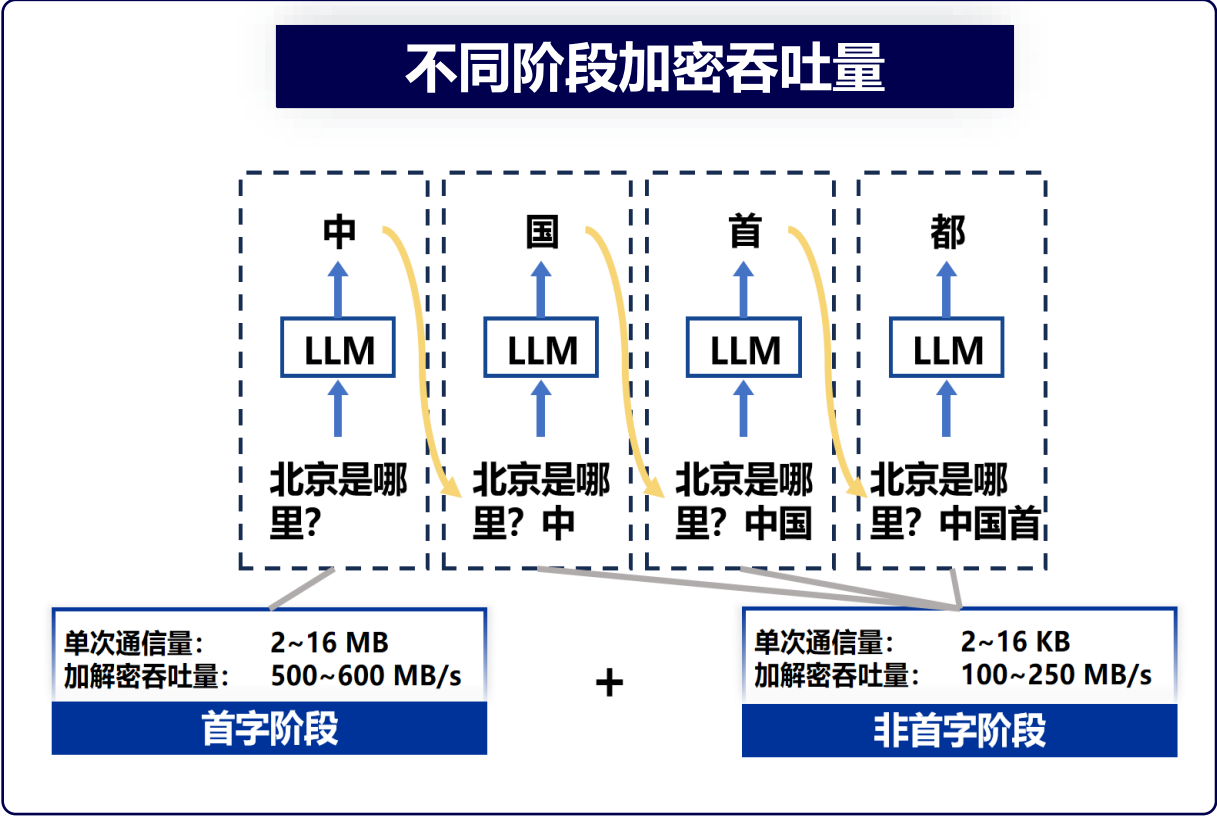
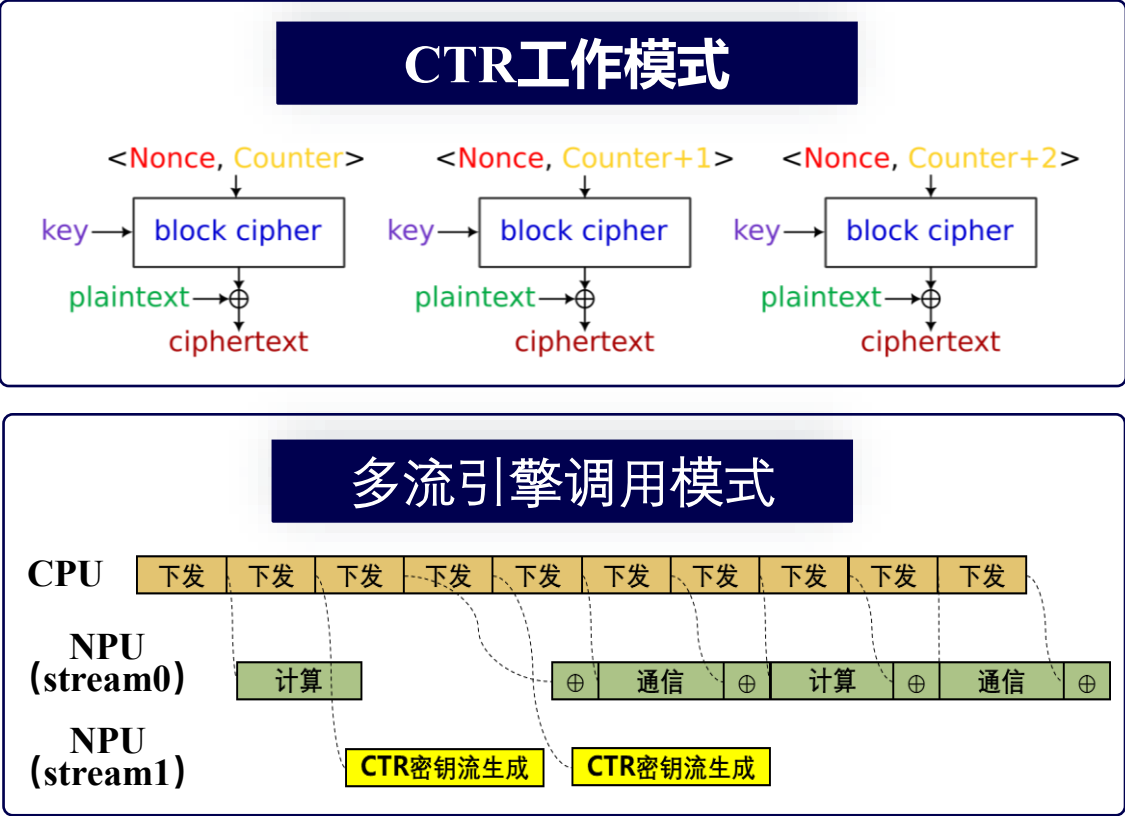
标量运算	2G
VS	
向量运算	256G
VS	
矩阵运算	8T

流水线路径优化设计



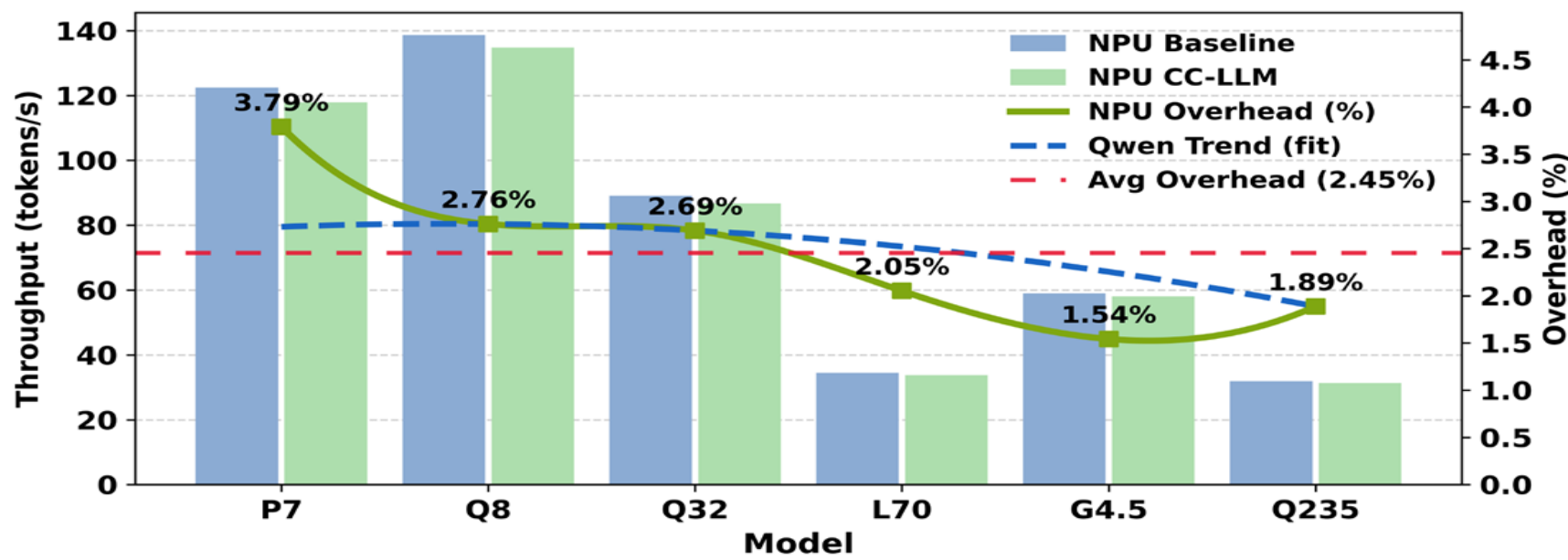
大模型计算与加密运算的协同优化

- CTR加密运算与大模型训推框架协同调度，挖掘**大模型闲置算力**生成密钥流
- 根据大模型推训不同阶段数据通信量与通信频率，**按需自适应生成密钥流**



针对昇腾的大模型训推系统加密优化

- 昇腾处理器端到端大模型推理加解密性能损失从15%下降到1.89%（Qwen3-235B@8卡910B）
- 已完成最新昇腾NPU架构（950DT）下联测适配
- 相关代码计划在CANN Crypto SiG开源



目录

Part 1 AI时代需要保护什么?

Part 2 密码学简史

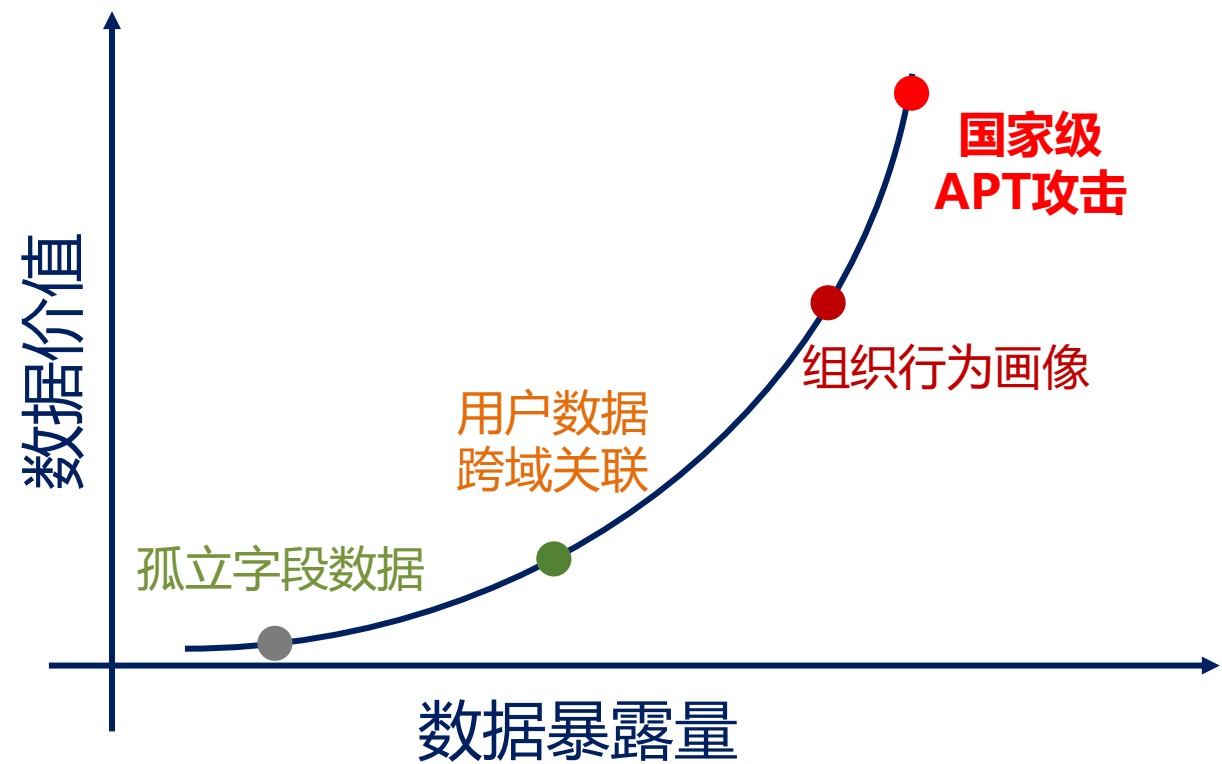
Part 3 Crypto4AI – TEE

Part 4 Crypto4AI – FHE

Part 5 Crypto4AI – ZKP

Part 6 密码学SIG现状及规划

- 智能计算时代，有何不同？
 - ▣ 数据的泄露所带来的问题变得尤为重要



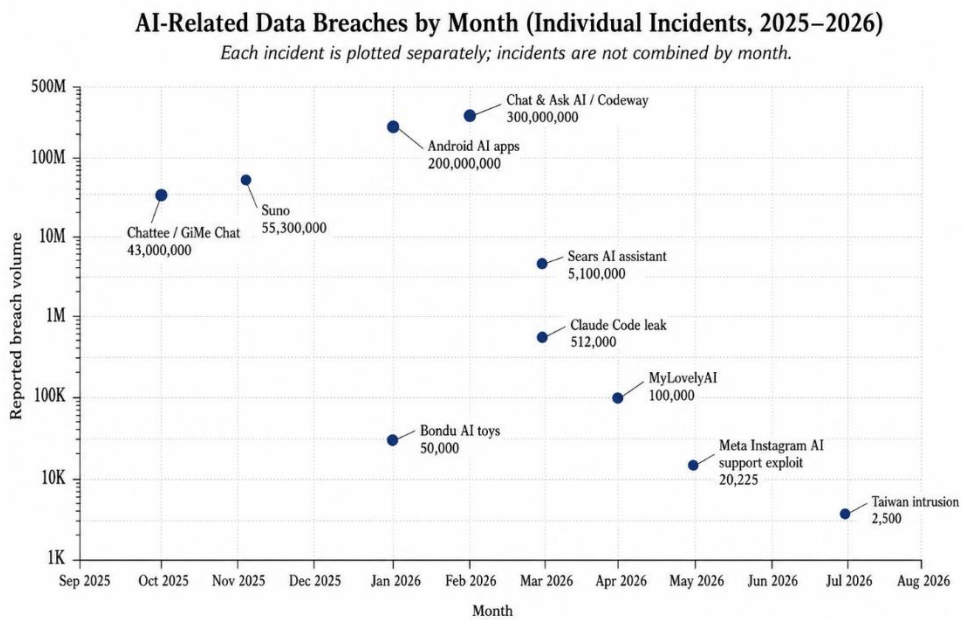
World ▾ Business ▾ Markets ▾ Sustainability ▾ Legal ▾ Commentary ▾ Technology ▾

AI-related data breaches surging, Verizon report says

By David Shepardson

May 20, 2026 1:06 AM GMT+8 · Updated May 20, 2026

AI生产漏洞导致的数据泄露首次超过凭证窃取



一年内AI相关的数据泄露

隐私计算关键技术对比

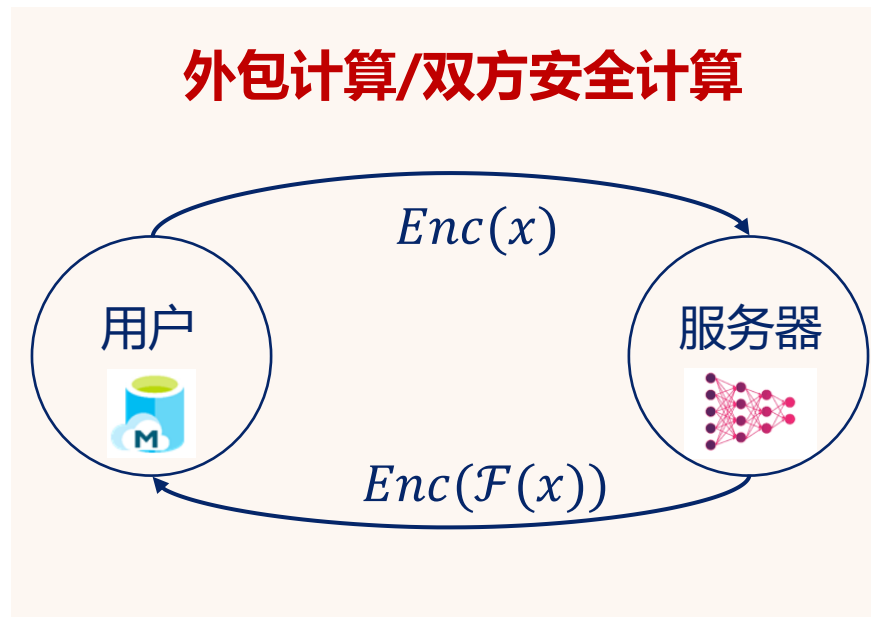
技术手段	性能			安全性
	计算速度	通信量	计算通用性	
差分隐私	极快	低	弱	噪声统计特性
可信执行环境	极快	低	强	硬件可信根
多方安全计算 (秘密共享, 零知识证明)	快	高	-	可证明安全
全同态加密	慢	低	强	可证明安全 (单一数学困难问题)

用户更倾向于使用可证明安全的协议

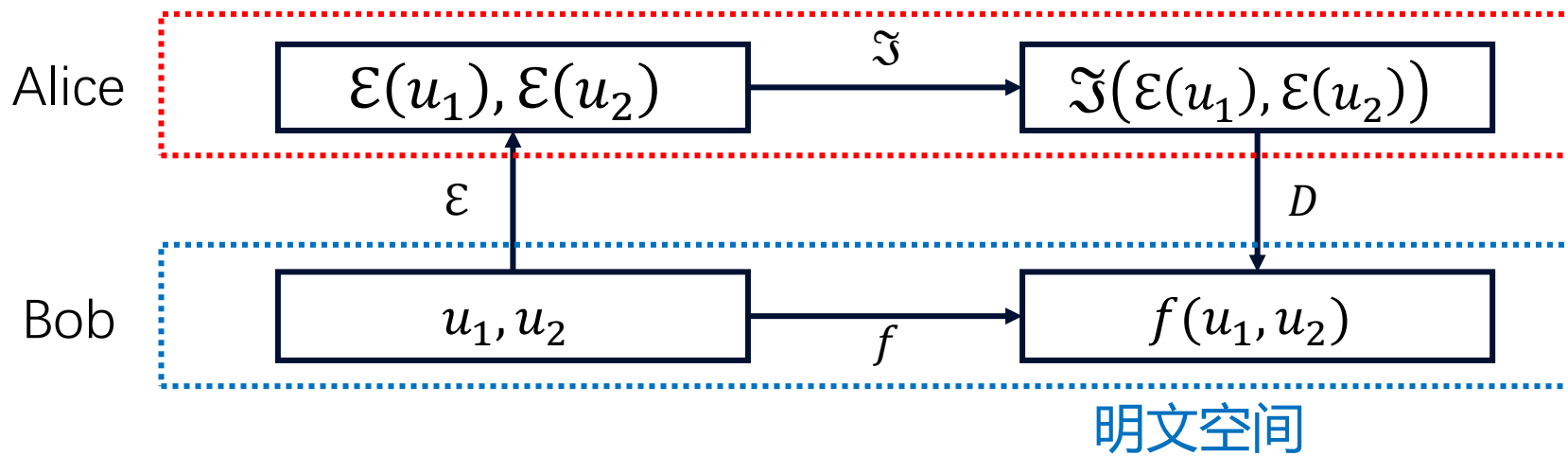


全同态加密

- 同态加密 (HE) :
对密文直接进行计算
- 优势：最优轮数、低通信、高安全
- 瓶颈：计算量



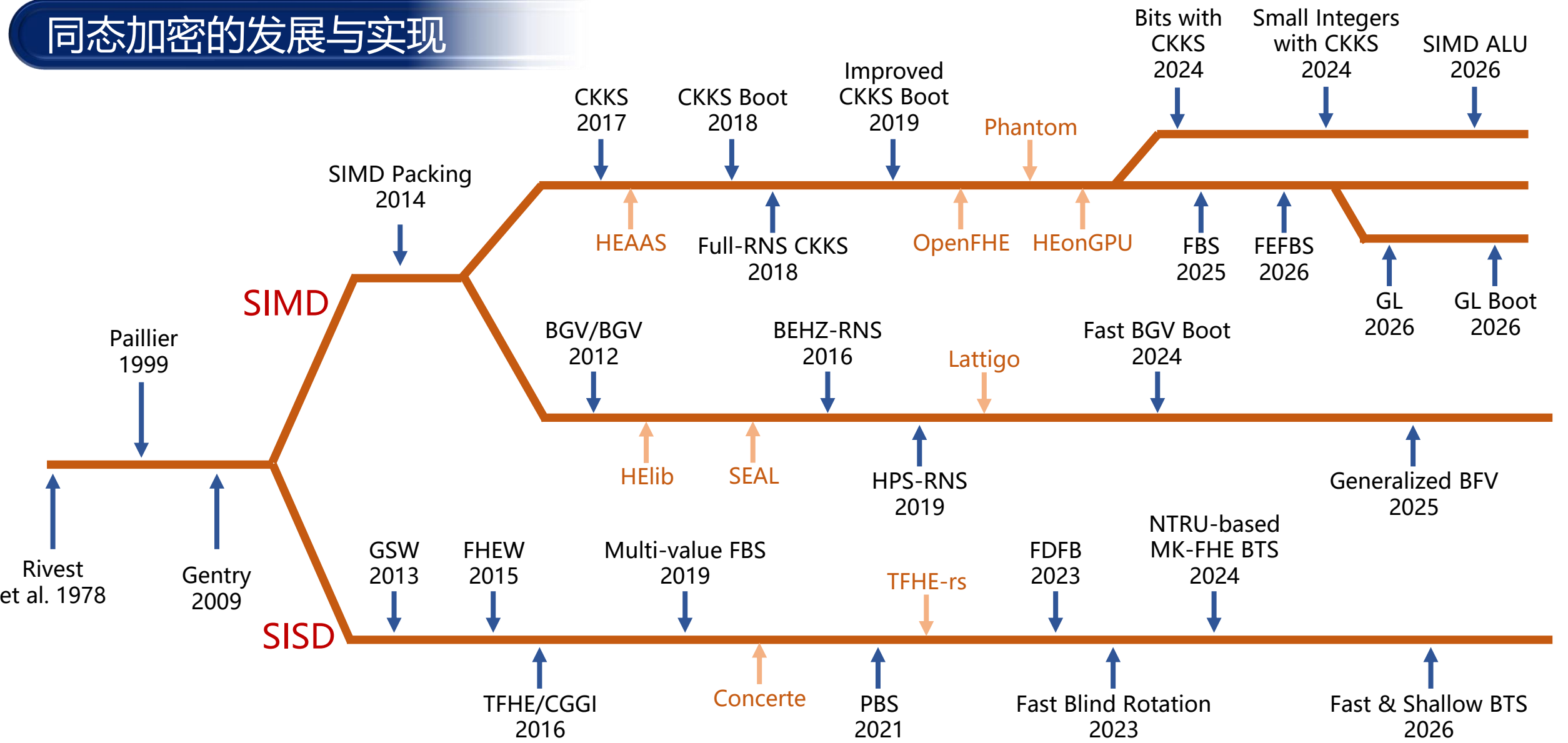
同态加密 (Homomorphic Encryption)



- 若 f 存在, $(\mathcal{E}, D)$ 即为 f -同态加密
 - 若 f 为图灵完备, $(\mathcal{E}, D)$ 为全同态加密
- 目前绝大多数全同态加密都基于格密码
 - 理想格 (Gen09)、整格 (LWE)、模格 (RLWE)、NTRU 格 (FINAL)

同态加密引言

同态加密的发展与实现



全同态加密的纵向发展

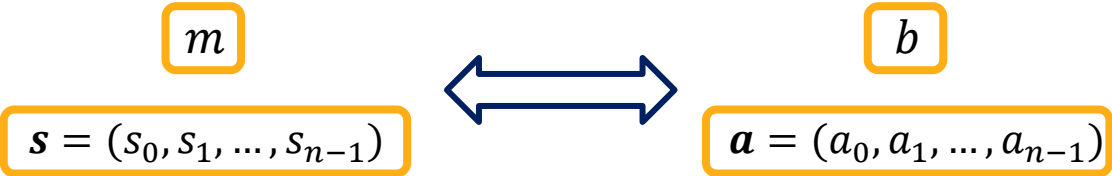


抽象层	主要任务	技术路线	技术方案
应用层	面向应用设计 专用 全同态算法	全加密数据库	部分同态: SAGMA、... 全同态: HEDA、...
		加密机器学习+ 数据分析	部分同态: FATE (微众)、Cheetah (Facebook)、... 全同态: 神经网络、决策树、...
		全加密处理器	全同态: VSP、FURISC、...
编译层	将非密态计算程序 编译 为密态计算程序	层级同态	CHET、EVA (微软)、...
		代数全同态	HEaaN.MLIR (首尔大学)、...
		逻辑全同态	Transpiler (谷歌)、...
实现层	实现 同态算子	层级同态	SEAL (微软)、PALISADE (MIT)、...
		代数全同态	HElib (IBM)、OpenFHE (国际)、...
		逻辑全同态	TFHE (Zama/鲁汶)、...
实现层	实现 底层数学计算	CPU	HEXL (英特尔)、HLG (华为)、...
		GPU	cuHE (微软)、...
		FPGA/ASIC	HEXL-FPGA (英特尔)、CraterLake (SRI、IBM)、...

同态加密的主要数学困难问题

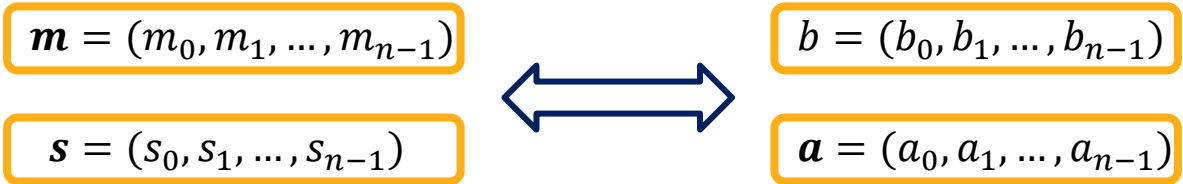
容错学习问题 (LWE, Learning with error)

- 模 q 剩余类 $Z_q = \{0, 1, \dots, q - 1\}$
- 明文模数 p , 密文模数 q
- 明文 $m \in Z_p$, 密钥 $s \in Z_q^n$, 高斯噪声 e , 随机向量 $a \in Z_q^n$
- 加密: $(a, b = -a \cdot s + \Delta m + e)$, $\Delta = q/p$ (BFV)
- 解密: $\lfloor (b + a^T s) / \Delta \rfloor = m$



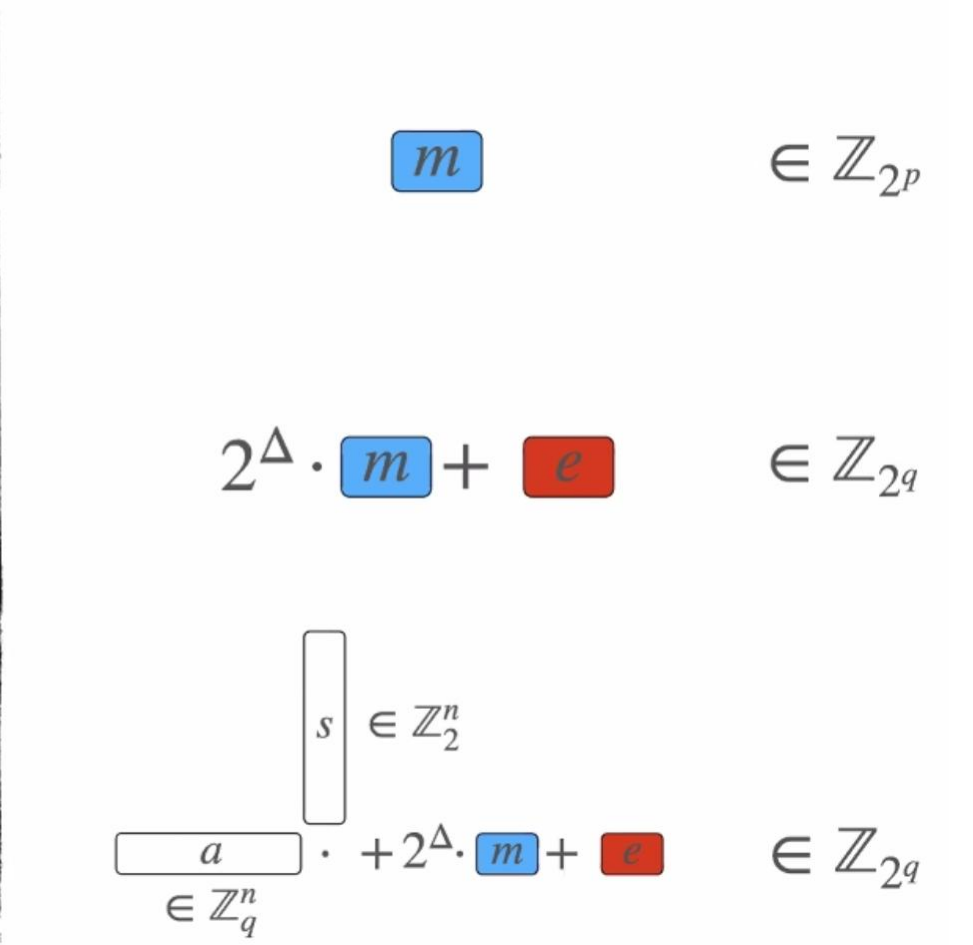
环容错学习问题 (RLWE, Ring learning with errors)

- 模 q 分圆多项式环 $R_{n,q} = Z_q[x]/(X^n + 1)$
- 明文模数 p , 密文模数 q
- 明文 $m \in R_{n,p}$ 密钥 $s \in R_{n,q}$ 高斯噪声 e , 随机 $a \in R_{n,q}$
- 加密: $(a, b) = (a, -a * s + \Delta m + e)$, $\Delta = q/p$
- 解密: $\lfloor (b + a * s) / \Delta \rfloor = m$



■ ■ ■ ■ ■

例子：LWE的对称加密



$$2^\Delta \cdot \boxed{m} + \boxed{e} \in \mathbb{Z}_{2^q}$$

$$\begin{array}{c} s \\ \hline a \end{array} \cdot + 2^\Delta \cdot m + e \in \mathbb{Z}_{2^q}$$

LWE密文的加法同态性

➤ 加法同态性

$$\begin{array}{l} (a_0, b_0 = -a_0 \cdot s + \Delta m_0 + e_0) \\ (a_1, b_1 = -a_1 \cdot s + \Delta m_1 + e_1) \end{array} \Rightarrow (a_0 + a_1, b_0 + b_1 = -(a_0 + a_1) \cdot s + \Delta(m_1 + m_0) + e_0 + e_1)$$

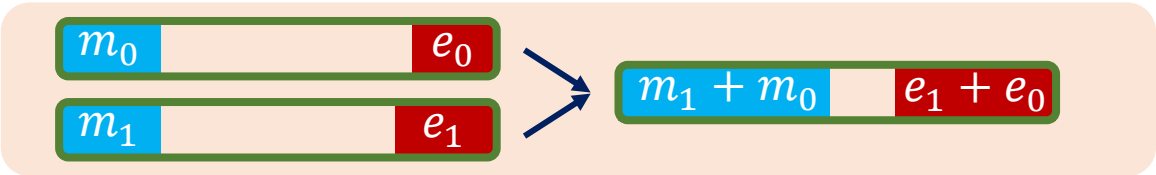
密文的和和的密文

噪声与密文自举

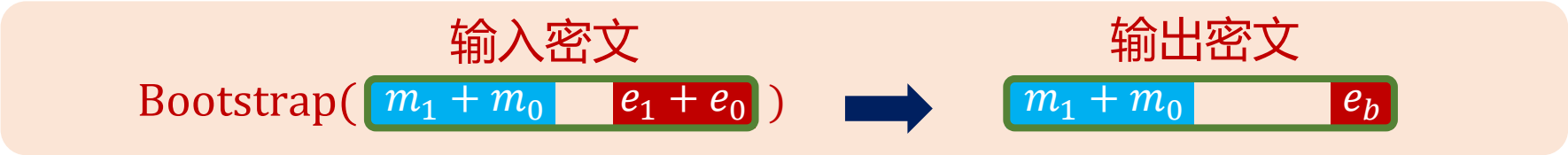
- 噪声是保护密文的重要组成部分
 - 然而会影响解密正确性
 - 同态操作会增加噪声

问题：
噪声过大导致
解密失败

$$\begin{aligned} (a_0, b_0 &= -a_0 \cdot s + \Delta m_0 + e_0) \\ (a_1, b_1 &= -a_1 \cdot s + \Delta m_1 + e_1) \end{aligned} \Rightarrow (a_0 + a_1, b_0 + b_1 = -(a_0 + a_1) \cdot s + \Delta(m_1 + m_0) + e_0 + e_1)$$



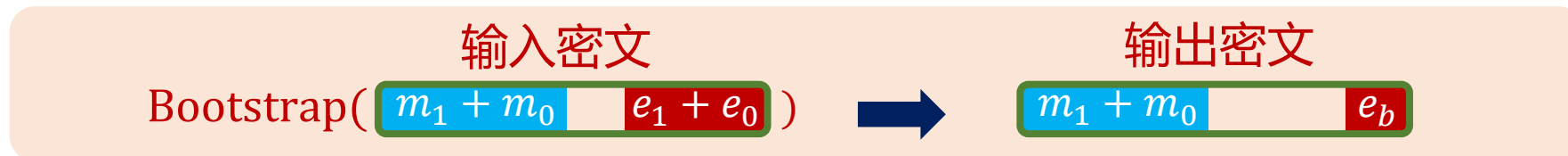
- 解决方案：密文自举



RLWE的密文自举

- 自举是全同态加密最关键的组成部分

- 自举的本质是让输出噪声与输入噪声解耦



- 自举是全同态加密最耗时的部分

- 早期：单比特自举数十**分钟级**

- 目前：单比特自举**微秒~毫秒级**

↓ 10^6

- 大量优化算法

- SIMD、RNS、多项式逼近、ExternalProduct、...

同态加密理论与应用

- 新型同态加密方案
 - ▣ 高性能自举
- 全密态智能
 - 高性能密态大模型

同态计算的可靠性与可信性

- 同态运算增大密文误差
 - ▣ 如何定义密文计算精度（如明文floating point的IEEE 754）？
- vFHE：可验证同态加密
 - ▣ 将FHE与ZKP/TEE/MPC融合

目录

Part 1 AI时代需要保护什么?

Part 2 密码学简史

Part 3 Crypto4AI – TEE

Part 4 Crypto4AI – FHE

Part 5 Crypto4AI – ZKP

Part 6 密码学SIG现状及规划

AI 越强大，信任越稀缺



当 AI 的能力被不断放大，我们却越来越难以回答一个问题：

模型是否被替换？

AI 服务商是否真的调用了宣称的模型？

推理能否复现？

这个结果是怎么算出来的？能被别人独立验证吗？

数据交给谁？

多方协作时，我的训练数据会不会被泄露？

核心矛盾 算力可以租，信任没法买



解决 AI 信任危机，需要同时回答两个独立的问题：

ZKP

零知识证明

证明计算做对了，但不泄露输入、模型与中间细节

解决「可验证」



MPC

安全多方计算

多方共同完成计算，每方只知道自己该知道的信息

解决「隐私保护」

ZKP 是什么？——数独直觉与「盲化信封」协议

我能证明我解出了这道数独，
但不需要把答案给你看。

	7				5		6	
9					3			8
		6				1		
	6	9	7	5		2		
4	5			3	2	8		
7	2			4	1	3		
6	3		8	2	4	5	1	9
2	9	5		1	6	4	8	7
8	1	4		9	7		3	

零知识证明：证明者 说服 验证者 命题为真，除此之外验证者什么也学不到。

AI 场景：证明「模型确实算出了这个输出」，但不必暴露参数与原始数据。

三步协议：不泄露答案的验证

- 1

承诺 Commitment

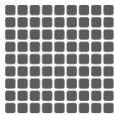
答案 1-9 随机置换 (1→G, 2→B...)，81 个格子全部盖上信封
- 2

挑战 Challenge

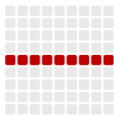
验证者随机三选一：查某一行 / 某一列 / 某个宫格，无法预判
- 3

验证 Verification

拆开 9 个信封、打乱查看：恰好 1-9 无重复——但看不出原坐标



9 × 9 = 81 封



随机指定第 5 行

G B E A C I D F H

✓ 9 个字母互不相同
洗牌后坐标信息已丢失

信封 = 哈希 / Pedersen 承诺 · 随机抽查 = 验证者挑战 · 重复 N 次：作弊率 $(8/9)^N \rightarrow 0$

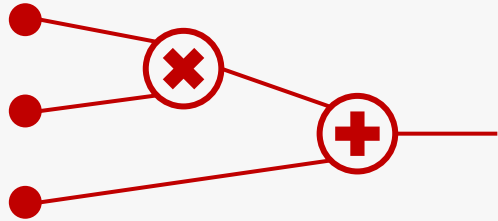
验证者 100% 确信我解出了数独，却始终不知道任何一格填了什么。



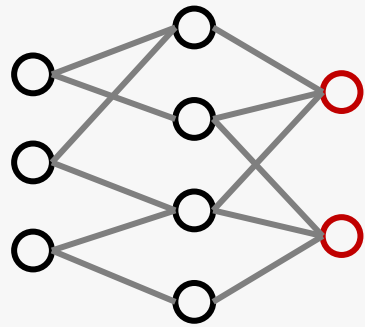
从数独到任意计算：一切都是电路

	7				5		6	
9					3			8
		6				1		
	6	9	7	5		2		
4	5			3	2	8		
7	2			4	1	3		
6	3		8	2	4	5	1	9
2	9	5		1	6	4	8	7
8	1	4		9	7		3	

数独
27 条约束 (行 / 列 / 宫)



加法门x乘法门任意计算
算术电路
ZKP 的通用「证明语言」



神经网络
一张巨大的算术电路

数独 = 一组约束

每行 / 每列 / 每宫 1-9 各出现一次。证明「我解出了」
= 证明「满足全部约束」。

任意计算 = 算术电路

加法门 + 乘法门是通用语言，任何程序都能编译成电路——这就是 zk-SNARK 的实际入口。

神经网络 = 巨大电路

矩阵乘 = 海量乘法门；ReLU / Softmax 等非线性需位分解模拟，**特别贵**。

「证明 AI 算对了」 = 「证明正确执行了这个电路」——一次推理动辄数百万乃至上亿约束，产生了非常庞大的计算量。

ZKP: 三个关键性质与 AI 推理验证



Shafi Goldwasser
MIT / Weizmann Institute
奠定了零知识证明的
理论基础

2012 ACM Turing Award



Silvio Micali
MIT
交互证明与密码协议的
奠基人之一

2012 ACM Turing Award

三大关键性质



完备性 Completeness

真命题一定能被证明：诚实的证明者总能说服诚实的验证者。

可靠性 Soundness

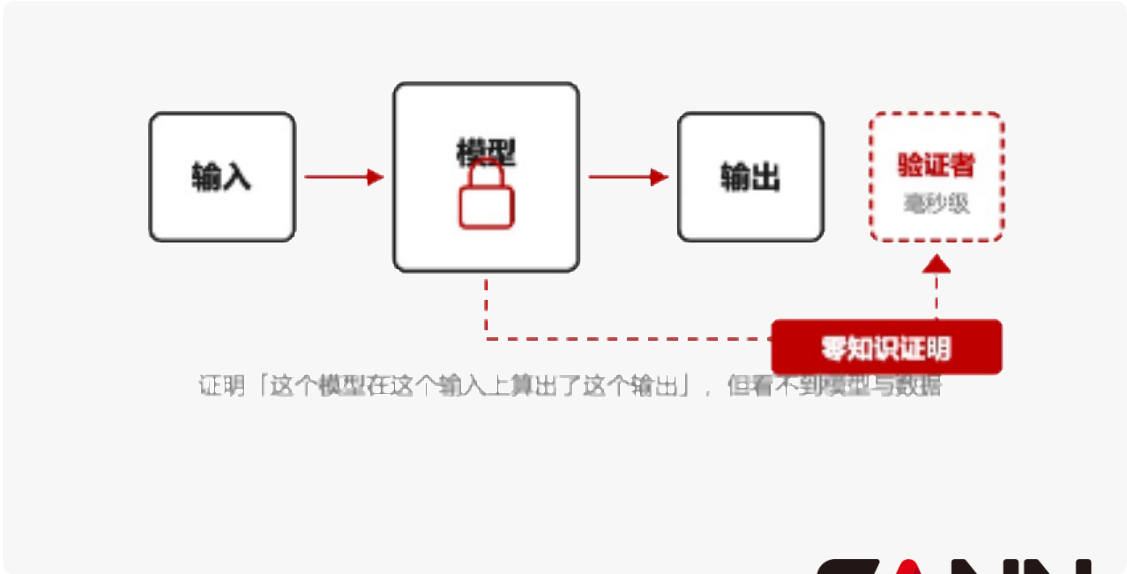
假命题无法被证明：作弊者不能让验证者相信错误结论。

零知识性 Zero-Knowledge

验证者除了「命题为真」，学不到任何额外信息。

ZKP for AI: 推理验证三步

- 01 编码成电路**
把推理的每一步（矩阵乘、激活函数）翻译成密码学电路。
- 02 生成证明**
Prover 运行模型并生成简短证明，绑定输入、模型与输出。
- 03 快速验证**
Verifier 毫秒级验证，无需重跑模型，也看不到模型细节。



ZKP 的挑战：证明很贵，但这正是 NPU/GPU 的机会

ZKP 从 1985 年的理论概念，走到 2020 年代的 zkML——四十年间，它从“数学游戏”变成了“让 AI 可验证”的工程工具。

三大痛点

证明生成慢

为一次 LLM 推理生成证明，往往比推理本身慢几个数量级。

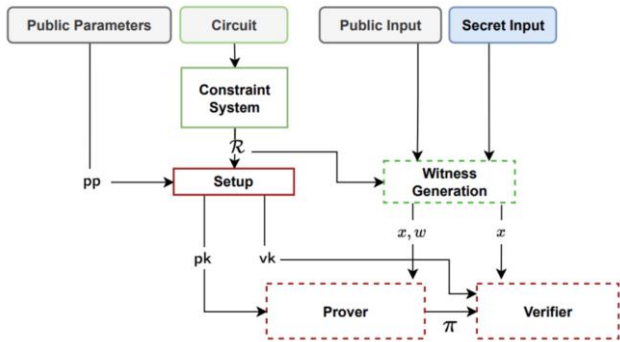
电路规模巨大

一次推理 = 数百万到上亿个约束，内存与计算双压力。

非线性算子昂贵

ReLU / Softmax 在电路里要靠位分解模拟，代价极高。

NPU/GPU 的用武之地



- 证明系统底层是海量多项式运算与 FFT/NTT
- MSM（多点标量乘）是典型的天生可并行负载
- 大整数与有限域运算的内存规模线性增长

密码学，就是一种新的计算负载 —— 而你们最懂加速



ZKP 主流技术路线

技术选型维度：

证明体积 Proof Size

验证速度 Verification Time

证明者开销 Prover Overhead

SNARKs

Groth16 · PlonK · Halo2

- ✓ 证明体积极小（几百字节）
- ✓ 验证极快（几毫秒）

NPU/GPU 瓶颈

MSM（多标量乘法）+ NTT（数论变换），极度吃显存带宽

STARKs

StarkNet · Plonky2

- ✓ 无需可信设置（No Trusted Setup）
- ✓ 抗量子攻击

NPU/GPU 瓶颈

底层大量哈希计算（Poseidon / Keccak）+ 多项式插值，适合高并发逻辑吞吐

Folding Schemes

对 AI 最关键

Nova · HyperNova

- ✓ 针对循环 / 迭代计算（Recurrent Computation）
- ✓ 大幅降低大模型的证明开销

NPU/GPU 瓶颈

LLM 的 Transformer 多层结构 + 自回归生成本质上就是循环计算，折叠方案天然适配



MPC 的起源：百万富翁问题

1982：两个百万富翁想比一比谁更富有

但他们都不想透露自己具体有多少钱。有没有一种协议，让他们在互不知道对方财富的情况下，判断谁的财富更多？

这就是姚期智提出的 百万富翁问题——安全多方计算（MPC）的理论起点。

同一个思想，更生活化的版本

五个同事想知道平均工资，但谁都不想暴露自己的工资。

没有可信第三方、不互相透露工资，还能算出正确的平均值吗？

MPC：可以

安全多方计算：多方一起算一个函数，每个人都只知道自己该知道的。

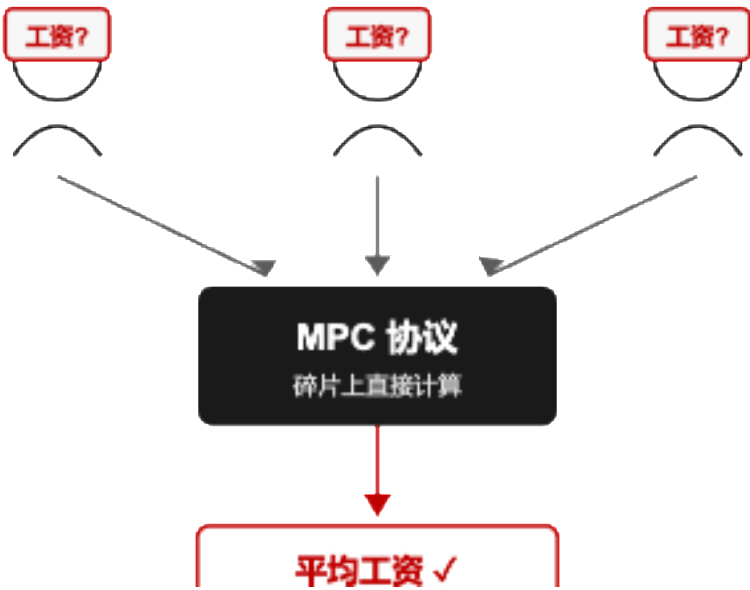


姚期智 Andrew Yao

清华大学 / Princeton

2000 ACM Turing Award

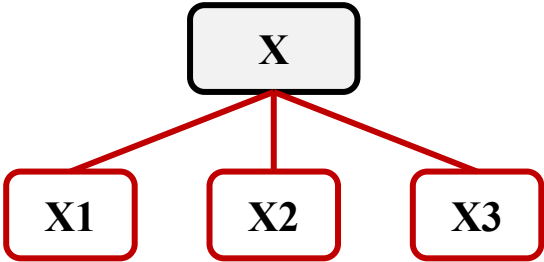
提出百万富翁问题、混淆电路（Garbled Circuits），奠定 MPC 理论基础



MPC 如何工作：秘密分享

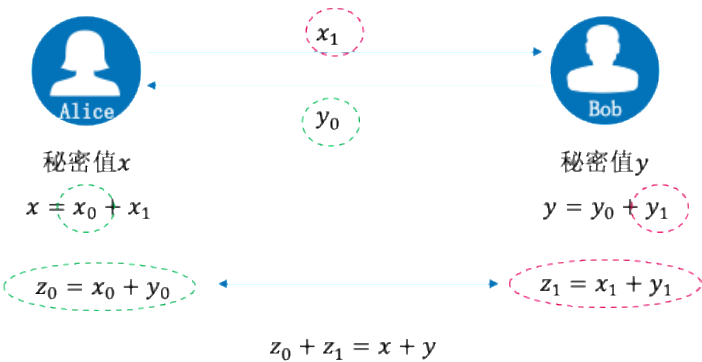
① 秘密拆分

把每个数拆成随机碎片，单看一个碎片毫无信息量。



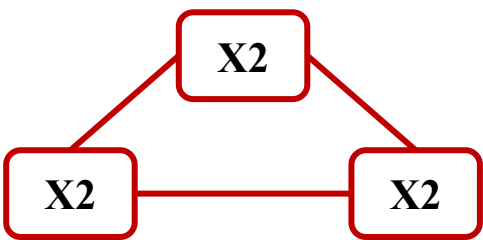
$X = X1 + X2 + X3$
任何单个碎片=随机数

加法运算



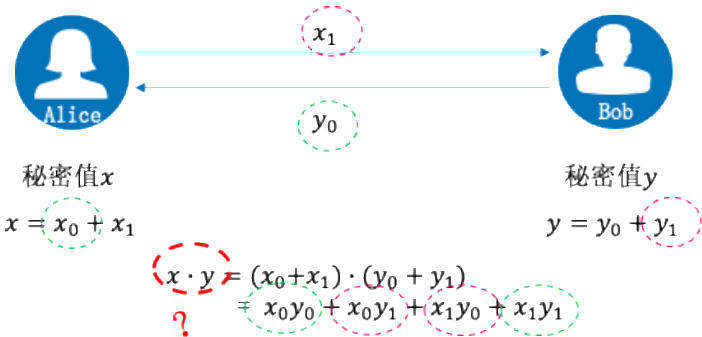
② 碎片上计算

各方持有碎片，通过协议直接在碎片上做加法、乘法。



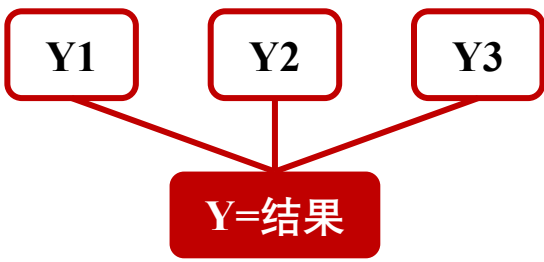
协议交互计算
分片之间做加法和乘法

乘法运算



③ 重构结果

汇总足够多的碎片，拼出最终结果——也只拼得出结果。



恢复每个分片的计算结果
只输出最终计算结果

MPC for AI: 数据不出域的两类场景



隐私联合建模

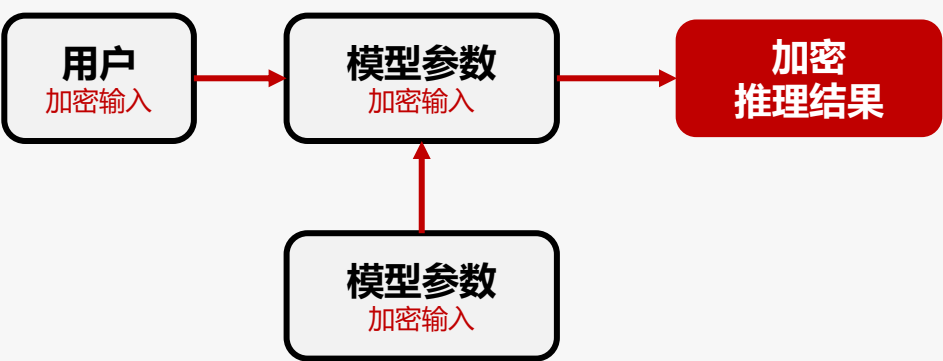
多家医院 / 银行 / 平台在各自数据不出域的前提下，联合训练同一个模型。



典型收益：小机构也能用上大样本，合规且安全

隐私保护推理

用户数据加密后交给云端大模型，服务器完成推理却看不到任何输入。



典型收益：敏感数据也能享受云端大模型能力



MPC 的挑战：通信与非线性是两大瓶颈

三大痛点

通信量爆炸

碎片乘法需要多方交互，通信量比明文计算高几个数量级。

非线性算子昂贵

比较、ReLU、Softmax 需要多轮交互或位分解，代价极高。

延迟难实时

层层交互叠加，让实时推理与在线服务很吃力。

工程上的机会

秘密分享	混淆电路	GMW协议	混合范式	FSS
				
优点: - 易于实现 - 计算开销低	优点: - 功能通用 - 通信轮次少	优点: - 通用多方场景 - 易认证	优点: - 优势互补 - 场景适应	优点: - 剪发性强 - 通信轮次少
缺点: 通信频繁	缺点: 计算开销高	缺点: 通信开销大	缺点: 切换验证难	缺点: 密钥较大

- 批处理摊薄通信开销，吞吐量是关键指标
- 线性层（矩阵乘）本就是 NPU 最擅长的负载
- 通信调度与计算重叠，是典型的系统优化问题

又是一次系统工程的胜利 —— 优化者大有可为

CANN 开源社区的机遇：密码学是新的计算负载

ZKP 核心运算

- MSM 多点标量乘：数万点并行累加
- FFT / NTT：教科书级并行负载
- 大整数与有限域算术

MPC 核心运算

- 秘密分享矩阵乘：GEMM 换皮
- 截断与比较：批量化处理
- 通信与计算的重叠调度

全栈迁移

- 算子优化经验直接复用
- 分布式与流水线设计能力
- 开源生态缺人，欢迎入场

你们写过的每一个高性能 kernel，都可能让可信 AI 快上一分

让 AI 的计算有据可验，
让信任成为基础设施

CANN

目录

Part 1 AI时代需要保护什么?

Part 2 密码学简史

Part 3 Crypto4AI – TEE

Part 4 Crypto4AI – FHE

Part 5 Crypto4AI – ZKP

Part 6 密码学SIG现状及规划



围绕昇腾 NPU 打造高性能密码软件库，提供丰富的密码算子与算法实现。

功能目标

覆盖国密与国际主流算法，建设统一易用的密码算子与算法库。

性能目标

面向高吞吐、低时延场景，发挥昇腾 NPU 并行计算能力，相对 CPU 基线显著加速。

生态目标

通过开源共建完善文档、测试与示例工程，沉淀密码算子开发规范与最佳实践。

自主目标

面向国密合规与信创适配，建设自主可控、可审计、可复用的密码计算能力。

扫码进入 SIG 主页



[gitee.com/cann/community
/CANN/signs/crypto](https://gitee.com/cann/community/CANN/signs/crypto)

工作目标
成员名单

SIG 已成立

工作目标与成员名单已在社区公开。

主仓建设中

密码库主仓 gitcode.com/cann/crypto 正在准备，
开放后第一时间在 SIG 主页公布。

算子开发

密码算子、测试用例、示例工程与文档

扫码进 SIG 主页



工作目标 · 成员 · 进展，持续更新

直接联系 Maintainer

程皓 hao.cheng@sdu.edu.cn

鞠雷 julei@sdu.edu.cn

彭聪 cpeng@whu.edu.cn

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：以开源驱动技术创新，降低开发门槛，聚力构筑高性能
昇腾 AI 算力底座

Vision: Building an Open, Easy-to-Use, and Technology-leading AI
Computing Ecosystem

Mission: Drive technological innovation through open source, lower
development barriers, and jointly build a high-performance Ascend AI
computing infrastructure.



上CANN社区获取干货



关注CANN公众号获取资讯

CANN